

FINAL PROJECT REPORT
ON
<EXPENSE TRACKER SYSTEM>



Department Of Computer Science
Microtek College of Management Technology
Barai, Nayepur, Baragaon, Varanasi, Uttar Pradesh

MAY, 2026



Mahatma Gandhi Kashi Vidyapeeth, Varanasi (U.P)

Under Guidance Of
Abhijit Sir

Submitted By:
NIVEDITA RAI (19724407023)
DURGESH CHAUBEY (19724407096)

DECLARATION

I hereby declare that the project entitled "**Expense Tracker System**" submitted to the Department of Computer Science, Microtek College of Management & Technology, in partial fulfilment of the requirements for the degree of Bachelor of Computer Applications (BCA), is my own original work carried out under the guidance of Abhijit Sir.

I further declare that this project has not been submitted, either in part or in full, to any other university or institution for the award of any degree or diploma. All information included in this report has been obtained and presented in accordance with academic rules and ethical conduct. I acknowledge that any violation of the above statement will result in severe disciplinary action.

Place: ____VARANASI_____

Date: _____

ACKNOWLEDGEMENT

It gives me immense pleasure to express my sincere gratitude to everyone who supported and guided me throughout the completion of this project.

First and foremost, I thank **God** for giving me the strength, patience, and determination to complete this project successfully.

I would like to express my heartfelt gratitude to my project guide, for continuous guidance, valuable suggestions, and constant encouragement during every stage of this project. Their support and feedback helped me improve my work and complete this project in a better and more meaningful way.

I am also sincerely thankful to the **Head of the Department**, the **Principal**, and all the faculty members of **Microtek College of Management & Technology** for providing the academic support, facilities, and positive learning environment needed for this project.

I would like to thank all my teachers whose knowledge and guidance throughout my BCA course helped me build the technical foundation required to undertake this project.

I am deeply grateful to my **family and friends** for their encouragement, understanding, and moral support during the long hours of development and documentation. Their support motivated me to keep going throughout this work.

Lastly, I would like to acknowledge the developers of **Python, Flask, SQLite3, Bootstrap, and Google Charts**, whose open-source technologies made this project possible.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this project. Even if not mentioned individually, their support and encouragement have been sincerely valued.

I humbly submit this project as the result of collective guidance, support, and effort, and I remain thankful to all who helped me in making it possible.

DURGESH CHAUBEY
19724407096

NIVEDITA RAI
19724407096

ABSTRACT

Managing personal finances has become increasingly difficult in today's digital world, where daily transactions happen through UPI payments, online shopping, subscriptions, and mobile wallets. Small expenses often go unnoticed, and over time they can affect a person's overall financial stability. Although many expense management tools are available, they are often either too basic, too costly, or too complicated for everyday users to use consistently.

This project presents the development of an **Expense Tracker SYSTEM**, a web-based application designed to help users manage their personal finances in a simple, secure, and effective way. The system is developed using **Python and Flask** and provides a platform where users can record income and expense transactions, organize them into categories, set monthly budgets, and monitor their spending habits.

A key feature of the system is its **budget alert mechanism**, which helps users avoid overspending by providing timely alerts when expenses approach predefined limits. In addition, the system includes an **analytics dashboard** that presents spending patterns and financial trends through visual charts, making financial information easier to understand.

Another important feature of the system is the analytics dashboard, which provides visual representation of spending patterns and financial trends through interactive charts and graphs. These analytical tools make it easier for users to understand where their money is being spent and identify areas where better financial planning may be needed.

The application stores user data securely using **SQLite3**, ensuring privacy and keeping financial information under the user's control. It also provides **report generation in PDF and CSV formats**, allowing users to maintain and download their financial records whenever needed.

The frontend of the application is developed using **HTML, CSS, and Bootstrap 5**, while **Google Charts** is used for data visualization. The system is designed to be responsive, user-friendly, and accessible across different devices.

This project demonstrates the practical implementation of personal expense management by combining expense tracking, budgeting, alerts, and analytics into a single platform. It provides a simple, private, and cost-effective solution for students, professionals, and everyday users who want better control over their finances.

INDEX	
TOPIC	Page No
Certificate (will be provided from college)	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
Index	v
1: Introduction	5-7 pages
1.1 Background of the Project	
1.2 Problem Statement	
1.3 Objectives of the Project	
1.4 Scope of the Project	
1.5 Limitations	
2: System Analysis	5-7 pages
2.1 Existing System	
2.2 Drawbacks of Existing System	
2.3 Proposed System	
2.4 Advantages of Proposed System	
2.5 Feasibility Study (Technical, Economic, Operational)	
3: System Design	10-15 Pages
3.1 System Architecture	
3.2 Data Flow Diagram (DFD) Lvl 0,1,2	
3.3 Entity Relationship Diagram (ERD) (NO CLASS DIAGRAM)	
3.4 Flowcharts/Use Case Diagram	
3.5 Database Design (Tables & Schema)	
3.6 Input and Output Design	
4: Implementation	5-7 pages
4.1 Tools and Technologies Used	
4.2 Module Description	
4.3 Development Environment	
4.4 Algorithms (if any)	
5: Testing	4-8 pages
5.1 Testing Methods	

5.2 Test Cases	
5.3 Results of Testing	
6: Results and Discussion	4-8 pages
6.1 System Output	
6.2 Performance Analysis	
6.3 User Interface Description	
7: Conclusion and Future Scope	2-3 pages
7.1 Conclusion	
7.2 Future Enhancements	
8: Source Code	as per requirement
8.1 Code Structure	
8.2 Important Code Snippets	
8.3 Complete Source Code (in case of very long code opt out)	
9: Screenshots (System Output)	as per requirement
9.1 Home Page	
9.2 Login Page	
9.3 Dashboard	
9.4 Module-wise Screens	
9.5 Reports / Output Screens	
10: Bibliography	1 page

INTRODUCTION

BACKGROUND OF THE PROJECT

In today's fast-moving digital world, managing personal finances has become more difficult than it appears. Money is often spent in small amounts throughout the day through UPI payments, online shopping, mobile wallets, subscriptions, utility payments, and automatic deductions like EMIs. These transactions happen quickly and often go unnoticed, which makes it easy for people to lose track of their spending. By the end of the month, many individuals are left wondering where their money has gone.

This problem has become even more common with the rapid growth of digital payments, especially in India. While technology has made payments easier and faster, it has also increased the number and variety of financial transactions people handle every day. Without a proper system to monitor these expenses, managing personal finances can become confusing and difficult, even for people who try to budget carefully.

Traditional methods such as maintaining expense records in notebooks or spreadsheets can help to some extent, but they have limitations. They require regular manual updates, take time to maintain, and often only show what has already happened. They do not provide meaningful insights into spending habits, nor do they help users identify patterns or warn them before they exceed their budget.

The **Expense Tracker SYSTEM** project is developed to solve this problem by providing a simple and organized platform where users can record their income and expenses while also gaining useful insights from their financial data. The purpose of the system is not only to store transaction records but also to help users understand their spending behavior, track financial trends, and make better decisions about managing money.

Although some people still use notebooks, diaries, or spreadsheets to record expenses, these traditional methods have many limitations. They require continuous manual effort, can be difficult to maintain, and often only provide a record of past spending rather than helping users improve future financial decisions. They do not offer spending analysis, category-based insights, or warning systems that can help prevent overspending.

One of the major strengths of this project is its focus on financial awareness and proactive control. Users can set monthly spending limits for different categories such as food, travel, shopping, or bills. The system continuously monitors expenses and sends automatic email alerts when spending is close to or exceeds the set budget. This allows users to take action in time rather than realizing overspending only after it has happened.

The application is developed using **Python** and the **Flask** microframework, which was chosen because of its lightweight structure, simplicity, and flexibility. Flask makes it easier to develop and understand the working of the system, while useful extensions such as **Flask-SQLAlchemy**, **Flask-Login**, **Flask-WTF** support features like database management, user authentication, form validation, and alert notifications.

For storing data, the project uses **SQLite3**, which keeps all financial records on the user's own machine instead of external servers. This improves privacy and security while keeping the system simple to use. The user interface is designed using **Bootstrap 5** to provide a clean and responsive experience, and **Google Charts** is used to generate interactive graphs and visual reports that make expense analysis easier to understand.

Another important aspect of the project is data analysis and visualization. Simply storing expenses is not enough unless users can understand patterns in their spending. For this reason, the project includes charts and reports that visually represent income, expenses, category distribution, and financial trends. These visual insights make it easier for users to identify where money is being spent and where improvements can be made.

Unlike many commercial expense management applications that depend on subscriptions, limited free versions, or user data collection, this project is designed to be affordable, privacy-focused, and accessible. It offers practical features without unnecessary complexity and gives users full control over their own financial information.

Many existing tools are either too simple to be useful, too costly for regular users, or designed for markets that do not fully support Indian payment habits and financial behavior. This project aims to bridge that gap by offering a solution that is simple, secure, and intelligent enough to support everyday financial management.

This project also reflects the growing importance of using technology not just for convenience, but for better decision-making. In the same way digital tools have transformed communication, education, and banking, they can also help people manage their finances more responsibly. The Expense Tracker is developed with this broader idea in mind.

This situation creates the need for a smarter and more practical solution. The Expense Tracker project was developed with this purpose in mind. The idea behind the project is to provide users with a simple platform where they can manage income and expenses in an organized way while also gaining a better understanding of their financial behavior.

The **EXPENSE TRACKER SYSTEM** is developed to fill this gap. It combines expense tracking, budgeting, analytics, and automated alerts into a single system that helps users not only record their finances but also manage them more effectively. It aims to improve financial awareness, encourage better spending habits, and support smarter decision-making.

PROBLEM STATEMENT

Managing personal expenses has become a challenge for many people, even though spending money has never been easier. With UPI payments, online shopping, subscriptions, mobile wallets, and automatic deductions becoming part of daily life, money often gets spent in small amounts without much attention. Over time, these small expenses add up, and many people struggle to understand where their money is going or why they are unable to save.

This problem is especially common among students, young professionals, and working individuals who handle frequent digital transactions but do not have a proper system to track and manage them. In many cases, people want to budget their expenses, but they either do not have the right tools or find existing methods too difficult or inconvenient to use regularly.

Traditional methods like notebooks or spreadsheets can help record expenses, but they require manual effort, regular updates, and discipline to maintain. Many users start using them but stop after some time because the process feels repetitive and time-consuming. These methods also only record spending; they do not provide meaningful insights, budget warnings, or support for better financial planning.

Although many expense management applications are available, they also have limitations. Some offer only basic features, while others require paid subscriptions for useful tools. Many store data on external servers, which raises privacy concerns, and some are not designed to support Indian payment habits such as UPI-based transactions. Because of this, existing solutions often do not fully meet the needs of everyday users.

This creates a clear gap in personal finance management. There is a need for a system that is simple enough for everyday users, secure enough to protect financial data, and intelligent enough to provide meaningful insights rather than just recording transactions. Users need a tool that not only tracks income and expenses but also helps them understand spending patterns, manage budgets effectively, and receive warnings before overspending occurs.

As a result, there is a clear need for a system that is simple to use, secure, affordable, and practical for personal finance management. Users need more than just a tool to record expenses — they need a system that helps them understand spending patterns, manage budgets, and receive alerts before overspending happens.

This project addresses that problem by developing the **Expense Tracker SYSTEM**, a system that combines expense tracking, financial analysis, budget monitoring, and automated alerts in one platform. The aim is to provide users with a reliable and easy-to-use solution that improves financial awareness, supports better spending habits, and helps people make smarter financial decisions.

Objectives of the Project

The main objective of this project is to develop a simple, secure, and practical web application that helps users manage their personal finances more effectively. The project is designed not just as a tool for recording income and expenses, but as a system that helps users understand their spending habits, control budgets, and make better financial decisions.

To achieve this goal, the following objectives have been defined:

To develop a secure user login system that allows users to register and access their accounts safely while protecting their personal and financial data.

To create a transaction management system where users can add, view, update, and delete income and expense records in an organized and convenient way.

To provide budget planning features that allow users to set monthly spending limits for different categories and monitor whether they are staying within their budget.

To generate automatic alerts and notifications when spending reaches a certain limit, so users can take action before overspending occurs.

To develop an analytics dashboard that presents income and expense information through charts and reports, helping users understand spending patterns and financial trends.

To provide report generation and export features so users can download and maintain their financial records in formats such as PDF and CSV.

To build a responsive and easy-to-use interface that provides a smooth user experience across desktops, tablets, and mobile devices.

To ensure privacy and secure storage of data by keeping user financial information under the user's control and storing it securely using SQLite3.

To apply full-stack web development concepts through the integration of database management, backend logic, frontend design, and secure authentication in one complete system.

To promote financial awareness and better budgeting habits by giving users a tool that helps them track expenses, control spending, and plan finances more effectively. Beyond these technical objectives, the project also aims to provide real practical value to users. It should help users gain a clearer picture of where their money goes, avoid unnecessary expenses, and develop more disciplined financial habits over time. Rather than acting as just a record-keeping application, the system is intended to function as a personal finance support tool that actively helps users manage their money.

Scope of the Project

The scope of this project includes the design and development of a web-based application that helps users manage their personal finances in a simple and organized way. The project focuses on creating a practical system where users can record their income and expenses, monitor budgets, analyze spending patterns, and maintain better control over their financial activities.

This project mainly covers the core features needed for personal expense management. It includes a secure user account system where users can register, log in, and manage their financial information safely. Each user can maintain their own records privately within the system.

The project also includes a transaction management module that allows users to add, view, edit, and delete income and expense records. This helps users keep track of their daily financial activities in an organized manner. To make expense tracking more meaningful, the system supports category-based management, where expenses can be grouped under categories such as food, travel, shopping, and bills.

Another important part of the project scope is budget planning and monitoring. Users can set monthly spending limits for different categories and compare their expenses against those limits. The system also includes automated email alerts to notify users when their spending reaches or exceeds the defined budget, helping them take action before overspending becomes a problem.

The project further covers an analytics dashboard that provides charts and visual reports for analyzing spending patterns, income-versus-expense trends, and category-wise expenditure. These features help users understand their financial behavior more clearly and support better decision-making.

The scope also includes report generation features that allow users to export financial records in PDF and CSV formats for future reference. A responsive user interface is included so the application can be used smoothly on desktops, tablets, and mobile devices.

From a technical perspective, the project covers the use of Python and Flask for backend development, SQLite3 for data storage, Bootstrap 5 for frontend design, and Google Charts for visual analysis. The system is designed to keep user data secure and private by storing information locally under the user's control.

At the same time, certain features are intentionally outside the scope of this project. The project does not include direct bank or UPI integration, native mobile application development, shared or multi-user budgeting features, AI-based financial advice, or real-time currency conversion. These features are considered future enhancements and are not part of the current implementation.

Therefore, the scope of this project is focused on developing a simple, secure, and effective expense management system that covers essential features such as expense tracking, budgeting, analytics, alerts, and reporting, while keeping the project practical and manageable.

Limitations

Like every software system, this project also has certain limitations. These limitations arise because of the project scope, technology choices, and the fact that this is an academic-level implementation. While the system provides the essential features needed for expense tracking and financial management, there are some areas where it has constraints and can be improved in the future.

One limitation of the project is that **all transactions must be entered manually** by the user or imported through CSV files. The system does not automatically collect data from banks, UPI applications, or payment platforms. Because of this, the accuracy of expense records depends on how regularly and carefully the user updates transactions.

Another limitation is related to the database used in the project. The system uses **SQLite3**, which is suitable for single-user and small-scale applications, but it may not be ideal for handling a large number of users or high-concurrency environments. For larger deployments, a more scalable database would be required.

The project also depends on **email services for budget alerts**, which means notification delivery relies on SMTP configuration and external email services. In some cases, alerts may be delayed or affected by email service limitations.

From a security perspective, the project is developed mainly for academic purposes and does not include all advanced features needed for large-scale public deployment. Features such as HTTPS configuration, advanced access controls, and request rate limiting are outside the current implementation.

Another limitation is that **reporting features are basic** and mainly focused on monthly records. The system does not currently support advanced reporting options such as custom date range reports or more detailed financial summaries.

The application also does not support **offline usage**. It requires a running server or internet connection to access features and manage records, which may be a limitation in environments with unreliable connectivity.

The project currently supports **only a single currency setting**, and real-time currency conversion is not included. Users dealing with multiple currencies would need to manage conversions manually.

In addition, the system does not include **direct integration with banks or UPI platforms**, which means transactions are not automatically synchronized from external financial sources.

System Analysis

Existing System

Before developing the proposed system, it is important to understand the methods and tools that people currently use for managing personal expenses. Although many solutions already exist, most of them have certain limitations that make them less practical, less secure, or difficult for everyday users to use consistently. Studying these systems helps identify the gaps that the proposed project aims to address.

1. Spreadsheets (Excel / Google Sheets)

Spreadsheets are one of the most common tools used for personal expense management because they are free, easily available, and flexible. Users can record income and expenses, create budgets, and organize financial data according to their needs. For users with basic spreadsheet knowledge, they can be a useful starting point.

However, despite these benefits, spreadsheets have major limitations. Their biggest drawback is that they depend entirely on manual effort. Every transaction has to be entered by the user, records must be updated regularly, and formulas need to be maintained correctly. If updates are missed even for a few days, it becomes difficult to keep accurate track of spending.

However, spreadsheets depend completely on manual effort. Every transaction has to be entered regularly, formulas need to be managed correctly, and maintaining records requires discipline. If users stop updating the sheet for a few days, tracking becomes difficult. They also do not provide automatic alerts, proactive budget monitoring, or easy visual analysis, which limits their usefulness for long-term expense management.

2. Mobile Finance Applications

Many mobile applications are available today for expense tracking and budgeting. These apps usually offer a cleaner interface than spreadsheets and make recording expenses easier. Some also provide budgeting tools and basic financial summaries.

However, many of these applications also have limitations. In many cases, useful features such as advanced budgeting, detailed reports, or alerts are locked behind paid subscriptions, while free versions provide only limited functionality. This can reduce their usefulness for regular users who want a complete solution without extra cost.

However, many of these applications have limitations. Important features are often available only through paid subscriptions, while free versions offer restricted functionality. Some apps also raise privacy concerns because they collect user financial data. In addition, many applications are not fully designed around Indian payment habits such as UPI-based transactions, which reduces their practical usefulness for some users.

3. Cloud-Based Finance Platforms: Cloud-based platforms such as Mint and YNAB provide advanced features like automated categorization, budgeting tools, and detailed dashboards. These platforms are powerful and offer strong analytical support.

Their strength lies in automation and analysis. They can provide valuable insights into spending behavior and support better budgeting decisions.

However, these platforms also have several limitations. Many of them are designed around foreign banking systems and may not fully support Indian banks or UPI-based transactions. This creates practical difficulties for users who rely on Indian payment systems.

Despite these advantages, they are often designed for foreign banking systems and may not fully support Indian banks or UPI transactions. They also depend on storing data on external servers, which may create privacy concerns for users. In many cases, access to advanced features also requires paid subscriptions.

4. Enterprise Accounting Software

Software such as Tally and QuickBooks provides comprehensive accounting features and is widely used in businesses. These systems are effective for managing accounts, records, and financial reports at a business level.

However, they are not suitable for personal expense management. They can be complex, expensive, and difficult for ordinary users without accounting knowledge. Their features are often far beyond what an individual needs for daily budget tracking.

They provide comprehensive features and are highly effective in professional environments. However, they are generally designed for business accounting rather than personal expense management.

Another concern is data privacy. Since these platforms store information on external servers, users may have concerns about sharing personal financial records online. In addition, many advanced features are often available only through subscriptions, which may not be affordable or necessary for ordinary users seeking a simpler solution.

5. Manual Record-Keeping

Some people still manage expenses through notebooks or paper records. This method is simple, private, and does not require technology.]

While this method may work for very basic tracking, it has significant limitations. It can be time-consuming, prone to human error, and difficult to manage as transactions increase. There is no automation, no budget alerts, no analysis, and no easy way to identify spending patterns.

Most importantly, very few combine simplicity, budgeting, visualization, alerts, privacy, and support for everyday Indian financial behavior in a single practical system. This creates a gap between what users need and what current tools provide.

However, manual methods have many limitations. They are time-consuming, prone to errors, and do not support automation, reporting, alerts, or analysis. As the number of transactions increases, managing records manually becomes even more difficult.

Drawbacks of Existing System

After studying the existing systems, it can be seen that although many tools are available for expense management, most of them have certain limitations that reduce their usefulness for everyday users. While some tools are good for recording expenses, others provide budgeting or analysis features, but very few offer a complete and practical solution. As a result, users often face difficulties in managing their finances effectively.

One major drawback of existing systems is the **lack of proactive budget control**. Most tools only record expenses after they have already happened. They may show where money has been spent, but they do not warn users in advance when they are close to crossing a budget limit. This makes it difficult to prevent overspending.

Another common issue is the **dependence on manual effort**. Methods such as spreadsheets and manual record-keeping require users to enter transactions regularly and maintain records with discipline. Many users find this repetitive and stop updating their records consistently, which reduces the usefulness of the system.

A further drawback is **limited financial analysis**. Many systems simply display transaction records but do not provide meaningful insights into spending patterns, trends, or financial behavior. Without proper analysis or visual reports, users may find it difficult to understand where improvements are needed.

Privacy concerns are also a major issue in many digital applications. Some finance apps store user data on external servers or collect financial information for other purposes, which can make users uncomfortable about sharing sensitive personal data.

Another limitation is the use of **fixed or inflexible expense categories**. Many existing applications offer predefined categories that may not match the actual spending habits of different users. This reduces flexibility and can make expense tracking less accurate or less relevant.

Some systems also provide **limited support for Indian users**, especially international finance platforms that do not fully support Indian banking methods, UPI transactions, or local payment practices. This reduces their practical usefulness for many users.

In addition, many applications place important features behind **paid subscriptions**, such as detailed reports, advanced budgeting tools, or data export options. This can make useful features inaccessible to users who prefer free solutions.

Another drawback is **complexity**. Some accounting tools are designed for business use and may be too complicated for ordinary users who only want a simple way to manage daily expenses.

Finally, some systems depend heavily on **internet or cloud services**, which can be a limitation for users who prefer local data storage or may face connectivity issues.

Proposed System

The proposed system, **Expense Tracker System** is designed to make personal finance management simpler, smarter, and more practical for everyday users. Instead of just recording income and expenses, the system helps users understand where their money goes, manage budgets effectively, and make better financial decisions.

The application is developed using **Python and Flask**, with **SQLite3** used to store data securely on the user's system. The interface is built using **HTML, CSS, and Bootstrap 5** to provide a clean and user-friendly experience, while **Google Charts** is used to present spending data through visual reports and graphs.

The proposed system includes several important features. It provides a **secure user login system** to protect personal financial data. Users can **add, edit, and manage income and expense transactions** easily in one place. The system also allows users to **set monthly budgets for different categories** and monitor their spending against those limits.

A key feature of the system is **proactive budget alerts**. Instead of only showing expenses after money has been spent, the system warns users when they are close to reaching their budget limits, helping them control overspending in time.

The system also includes an **analytics dashboard** that shows spending patterns, category-wise expenses, and financial trends through interactive charts. This makes it easier for users to understand their habits and plan better. In addition, users can **export financial reports in PDF and CSV formats** whenever needed.

Unlike many traditional expense trackers, this system is not only a record-keeping tool but also a support system that provides insights and timely alerts. It is designed to be simple, private, and affordable, making it suitable for students, professionals, and everyday users. By combining expense tracking, budgeting, analytics, and alerts in one platform, the proposed system offers a practical solution for better personal financial management.

One of the major features of the proposed system is secure user authentication. Since financial data is sensitive, the system provides a login mechanism to protect user information and ensure that personal records remain private and accessible only to authorized users.

The proposed system can therefore be viewed not only as an expense tracking application but also as a financial support tool. It helps users move from simply recording expenses to actively managing money. By combining transaction management, budgeting, visual analysis, and automated alerts in one platform, the system provides a more complete solution to personal finance management problems.

Overall, the proposed system acts not just as an expense recorder, but as a helpful financial companion that supports users in managing money more effectively and making informed financial decisions.

Advantages of Proposed System

The proposed system offers several advantages that make it more practical and useful than many existing expense management tools. It is designed not only to record expenses, but also to help users manage money more effectively through privacy, insights, and proactive support.

One of the major advantages of the system is **data privacy and security**. All financial information is stored locally in the SQLite3 database, which means user data remains under the user's control. There is no third-party data sharing, no Behavioural tracking, and no risk of financial information being used for advertising purposes.

Another important advantage is **proactive budget alerting**. Unlike traditional systems that only show expenses after they occur, this system warns users when spending approaches predefined budget limits. This helps users take action early and avoid overspending.

The system also provides **meaningful analytics and visual insights** through interactive charts and reports. Users can easily understand spending patterns, track trends, and identify where most of their money is being spent. This makes financial planning easier and more informed.

A further advantage is **custom category management**, where users can create and manage expense categories according to their own lifestyle and spending habits. This provides greater flexibility than systems with fixed category structures.

The proposed system is also designed with a **simple and user-friendly interface**, making it easy to use even for people without technical or accounting knowledge. Users can manage their expenses without needing complex setup or training.

Another benefit is its **responsive design**, which allows the application to work smoothly on desktops, tablets, and mobile devices. This improves accessibility and convenience for users.

The system also supports **free report generation and export**, allowing users to download their financial records in PDF and CSV formats whenever needed. This ensures users have full access and control over their data.

In addition, the application has **no subscription cost or hidden charges**. All features are available to every user without premium plans, making it affordable for students and individual users.

Finally, the system is **lightweight and easy to maintain**, requiring minimal setup while still providing useful features for personal financial management.

Overall, the proposed system offers advantages in privacy, budgeting, analytics, flexibility, and affordability, making it a practical and effective solution for managing personal expenditure.

Feasibility Study

A feasibility study is an important part of any software project because it helps determine whether the proposed system can be developed successfully and whether it will be practical for users once implemented. Before starting development, it is necessary to evaluate the project from technical, economic, and operational perspectives. This helps ensure that the system is realistic, affordable, and capable of meeting user needs. Based on this analysis, the proposed Expense Tracker with Insightful Analytics is considered feasible for implementation.

For the proposed **Expense Tracker System**, the feasibility study has been carried out to evaluate whether the system can be implemented effectively using available resources and whether it can perform its intended purpose in real-life situations. Based on this analysis, the proposed system is considered feasible for development and practical for implementation.

Technical Feasibility

Technical feasibility refers to whether the proposed project can be developed using available technologies, tools, and technical resources. The proposed system is technically feasible because it is based on technologies that are reliable, widely used, and appropriate for web application development.

The proposed system is technically feasible because it is developed using technologies that are reliable, widely used, and suitable for web-based application development. The project uses **Python** as the programming language and **Flask** as the web framework, along with **SQLite3** for database management, **Bootstrap 5** for interface design, and **Google Charts** for data visualization. These technologies provide all the necessary tools required to build the proposed system effectively.

The technologies used are also well documented and supported by large developer communities, which makes development easier and reduces technical risks. Since these tools are commonly used in academic projects and practical development, they are already familiar and manageable during implementation.

Another reason the system is technically feasible is that it does not require specialized hardware, costly infrastructure, or advanced software environments. The application can be developed and executed on a standard computer system using commonly available resources. The technologies used are well documented, easy to learn, and already familiar in academic studies, which reduces technical challenges during development. Therefore, implementing the proposed system is practical and achievable.

Economic Feasibility

Economic feasibility refers to whether the project can be developed and implemented within reasonable cost and whether the benefits justify the expenses involved.

The proposed system is economically feasible because the cost involved in development and implementation is very low. Most of the technologies and tools used in this project are **free and open-source**, including Python, Flask, SQLite3, and Bootstrap. No paid software licenses, expensive APIs, or major infrastructure investments are required.

Compared with many commercial expense management applications that require subscriptions or charge users for advanced features, the proposed system provides useful functionality without recurring financial burden. This makes it not only affordable to develop but also practical and economical for end users.

The project can be developed using existing resources such as a personal computer and freely available development tools, which minimizes financial cost. Even for deployment, the system can operate without significant expenses. Compared with commercial finance applications that often require paid subscriptions, the proposed system provides useful features at little to no cost. This makes the project cost-effective for development as well as practical for end users.

Operational Feasibility

Operational feasibility refers to whether the proposed system will function effectively in real-world use and whether users will be able to adopt and operate it easily.

The proposed system is operationally feasible because it is designed to be simple, practical, and easy for users to adopt. The application allows users to record expenses, manage budgets, receive alerts, and view reports through a straightforward and user-friendly interface. It does not require technical knowledge or accounting expertise, which makes it accessible to a wide range of users.

Features such as proactive budget alerts, analytics dashboards, and downloadable reports improve usability and support day-to-day financial management. The system is intended for students, professionals, and general users, making it suitable for practical real-world use. Since the application focuses on simplicity and convenience, users can operate it effectively without special training.

The application does not require users to have advanced technical knowledge or accounting expertise. This makes it suitable for ordinary users such as students, working professionals, and households who need a simple way to manage personal finances.

Another factor supporting operational feasibility is that the system fits naturally into everyday financial activities. Users can enter transactions as they occur, monitor budgets regularly, and use alerts to avoid overspending. This makes the application practical for routine use rather than only occasional use.

SYSTEM DESIGN

System Architecture

The Expense Tracker System is designed using a three-tier client-server architecture, which is a common structure used in web applications. The system is divided into three main layers: the Presentation Layer, Business Logic Layer, and Data Layer. This separation helps keep the system organized, easy to manage, and efficient to maintain.

Presentation Layer (Frontend)

The Presentation Layer is the part of the system that users interact with directly through a web browser. It provides the interface where users can register, log in, add income and expenses, create budgets, and view reports.

This layer is built using **HTML5**, **CSS3**, **Bootstrap 5**, **JavaScript**, and **Jinja2 templates**. To make financial information easier to understand, **Google Charts** is used to display graphs and dashboard reports.

Business Logic Layer (Backend)

The Business Logic Layer is the main processing part of the system. It receives user requests, validates data, applies business rules, and processes the required operations.

The backend is developed using **Python Flask** and organized into different modules using **Flask Blueprints**, including:

- Authentication, Transactions, Budgets, Categories, Analytics Reports.

This layer handles important tasks such as managing user login, processing income and expenses, calculating budget usage, generating reports, sending email alerts, and providing data for charts. It acts as the connection between the user interface and the database.

Data Layer (Database)

The Data Layer stores all the information used by the system. The project uses **SQLite3** as the database and **SQLAlchemy ORM** for managing database operations.

The database stores records such as:

- User account details, Transaction information, Categories, Budget records, Reports and summaries

Using **SQLAlchemy** makes database operations simpler by allowing data to be handled through Python models instead of direct SQL queries. This keeps the code cleaner and easier to maintain.

Data Flow Diagram

A DFD is a fundamental tool in systems analysis that provides a graphical representation of the data flow within an information system. It clearly and systematically brings out how information is received, processed, stored, and ultimately distributed to its desired destinations. In the context of a Expense Tracker System, DFD is an essential modeling technique, highlighting the logical flow of data without digging into the technical or implementation-specific aspects of the system. It clarifies the boundary of the system and the interactions occurring within it through standardized symbols representing key system components, including external entities, internal processes, and data stores. This makes the modeling process especially valuable during early requirements analysis and system design, where the goal is to develop a common understanding of the system functionalities among the different stakeholders. DFDs will help analysts verify how inputs are transformed into outputs, identify the nature and role of data repositories, and understand the dependencies between the system, the expense tracker system, user, and admin. This clarity can be extended to effectively communicate with user and admin to validate whether the processes depicted correctly reflect the intended functions and operational needs of the system.

In presenting the DFD for the Expense Tracker System, both the overall structure and the levels of decomposition are to be described, allowing the logic of the system to be conveyed in greater detail. The most basic level, termed the Context Diagram or Level 0 DFD, represents an overview of the entire expense tracker as one unified process. At this level, only high-level interactions are shown, such as how users ask for transaction, what information the system provides in transaction, or how budget are added to its inventory. The Context Diagram clearly delineates system boundaries, showing which things reside within the system and which of these interactions emanate from entities outside it. Once this larger structure has been outlined, the DFD moves on to Level 1, wherein the single high-level process is decomposed into its major sub processes. For a Expense tracker System, these can include Expense Tracker System, both the overall structure and the different levels of decomposition are described so that the working logic of the system can be understood clearly and in greater detail. The Data Flow Diagram is used to represent how data moves between users, admin, databases, and internal processes of the system. It helps show how information enters the system, how it is processed, where it is stored, and what output is generated.

At this level, the **User** interacts with the system by providing inputs such as login credentials, requests to view expense records, and dashboard access requests. In return, the system provides outputs such as confirmation messages, access responses,

and customer dashboard information. Similarly, the **Admin** interacts with the system for managing records and receives confirmation status or system responses. The system also communicates with the **SMTP Server** for sending automated budget alerts through email notifications. At this stage, only the boundaries of the system are shown clearly, identifying what lies inside the Expense Tracker System and what entities exist outside it.

For this project, the DFD is represented in **three levels**: Level 0 DFD (Context Diagram), Level 1 DFD, Level 2 DFD

DFD Components (Symbols Used)

A DFD is made using four basic symbols.

1. External Entity

Symbol Used: Rectangle

Represents a person, system, or source outside the application that sends or receives data.

Examples in this project: User, Admin, SMTP Server

User

The user interacts with the system by: Registering Logging in Adding expenses
Setting budgets, Viewing reports

Example:

User sends: Login credentials, Expense data, Budget data

User receives: Dashboard Budget alerts Reports

Admin

The admin manages and monitors system operations.

Admin can: View user records , Manage transactions, Request reports, Monitor system status

SMTP Server

SMTP server is treated as an external entity because it handles outgoing email alerts.

Example: When budget exceeds 80%: System sends alert request to SMTP server.

SMTP sends email notification to user.

2. Process

Symbol Used: Circle

Processes represent activities where data is transformed or handled.

Examples in the project: User Management, Transaction Management, Budget Monitoring, Report Generation

User Management

Handles: Registration, Login, Profile validation

Example:

Input: Login credentials

Processing: Validate username and password

Output: Login confirmation

Transaction Management

Handles: Add transaction, Edit transaction, Delete transaction, View transaction

Example:

Input: Expense entry

Process: Store transaction

Output: Updated transaction record

Budget Monitoring

Handles: Budget checking, Expense comparison, Alert generation

Example:

Input: Expense data

Process: Compare with budget

Output: Budget alert

Report Generation

Handles: Reading transaction data, Generating PDF report, Exporting CSV report

3. Data Flow

Symbol Used: Arrow

Shows movement of data between components.

Examples: Login credentials, Expense data, Budget details, Transaction data, Alert email

Example:

User → Expense Data → Transaction Management

This means: User sends expense data to transaction management process.

4. Data Store

Symbol Used: Open-ended Rectangle / Parallel Lines

Represents where data is stored.

Examples: User Database, Transaction Database, Budget Database

User Database

Stores: User records, Passwords, Profiles

Transaction Database

Stores: Income entries, Expense entries, Transaction history

Budget Database

Stores: Budget limits, Alert flags, Category budgets

Level 0 DFD

Level 0 is the highest-level view of the system.

It treats the whole Expense Tracker as one single process.

In your diagram:

Main process: **Expense Tracker System**

External entities connected: User, Admin, SMTP Server

Level 0 Flow

User to System

User sends: Login/Credentials, Request/View Data

System returns: Customer Dashboard, Confirmation/Deny

Flow: User → Login Credentials → Expense Tracker System

Expense Tracker System → Dashboard → User

Admin to System

Admin sends: Manage Records

System returns: Confirmation Status

Flow: Admin → Manage Records → Expense Tracker System

Expense Tracker System → Confirmation Status → Admin

System to SMTP Server

System sends: Budget Alert Data

SMTP sends email alerts.

Flow: Expense Tracker System → Budget Alert System → SMTP Server

Purpose of Level 0

Shows: Overall system boundary, Major external actors, Basic inputs and outputs

It does not show internal processes.

Level 1 DFD

Level 1 breaks the main process into major sub-processes.

In your diagram, Expense Tracker System is divided into:

User Management, Transaction Management, Budget Monitoring, Report Generation

1. User Management Process

Handles: User login, Registration, Profile validation

Connected to: User Database

Flows: User → Login/Register, User Management → Validate User

User Database → User Details

2. Transaction Management

Handles: Add transaction, Store transaction, Update transactions

Connected to: Transaction Database

Flow: User → Expense Data, Transaction Management → Store/Update Transaction

Transaction Database → Transaction Data

3. Budget Monitoring

Handles: Read budget data, Check spending, Generate alerts

Connected to: Budget Database

Flow:

User → Budget Data, Budget Monitoring → Read Budget Data

Budget Database → Budget Details

4. Report Generation

Handles: Read transactions, Generate reports, Export PDF/CSV

Connected to: SMTP, Server Admin

Flow: Admin → Request Report, Report Generation → PDF/CSV Report
Admin receives report

Purpose of Level 1

Shows: Major internal processes, Data stores, Flow among processes

It gives more detail than Level 0.

Level 2 DFD

Level 2 breaks each Level 1 process into smaller detailed sub-processes.

Process 1.0 User Management

Sub-processes: 1.1 Register User, 1.2 Login User, 1.3 Manage User

1.1 Register User

Input: Registration user data

Process: Save user details

Output: Registration confirmation

1.2 Login User

Input: Login credentials

Process: Validate user

Output: User validated

1.3 Manage User

Input: Profile update

Process: Update user information

Output: Updated profile

Process 2.0 Transaction Management

Sub-processes: 2.1 Add Transaction, 2.2 View Transaction, 2.3 Edit Transaction
2.4 Delete Transaction, 2.5 Import Transaction CSV

Example:

User sends: Transactional details

Process 2.1: Add transaction

Stored in: Transaction Database

Process 3.0 Budget Monitoring

Sub-processes: 3.1 Set/Update Budget, 3.2 Check Budget Expenses, 3.3 Generate Alert

Example

User sets: Food Budget = ₹5000

User expenses: ₹4000

Process 3.3 triggers alert. SMTP sends warning email.

Process 4.0 Report Generation

Sub-processes: 4.1 Fetch Data., 2 Generate PDF Report

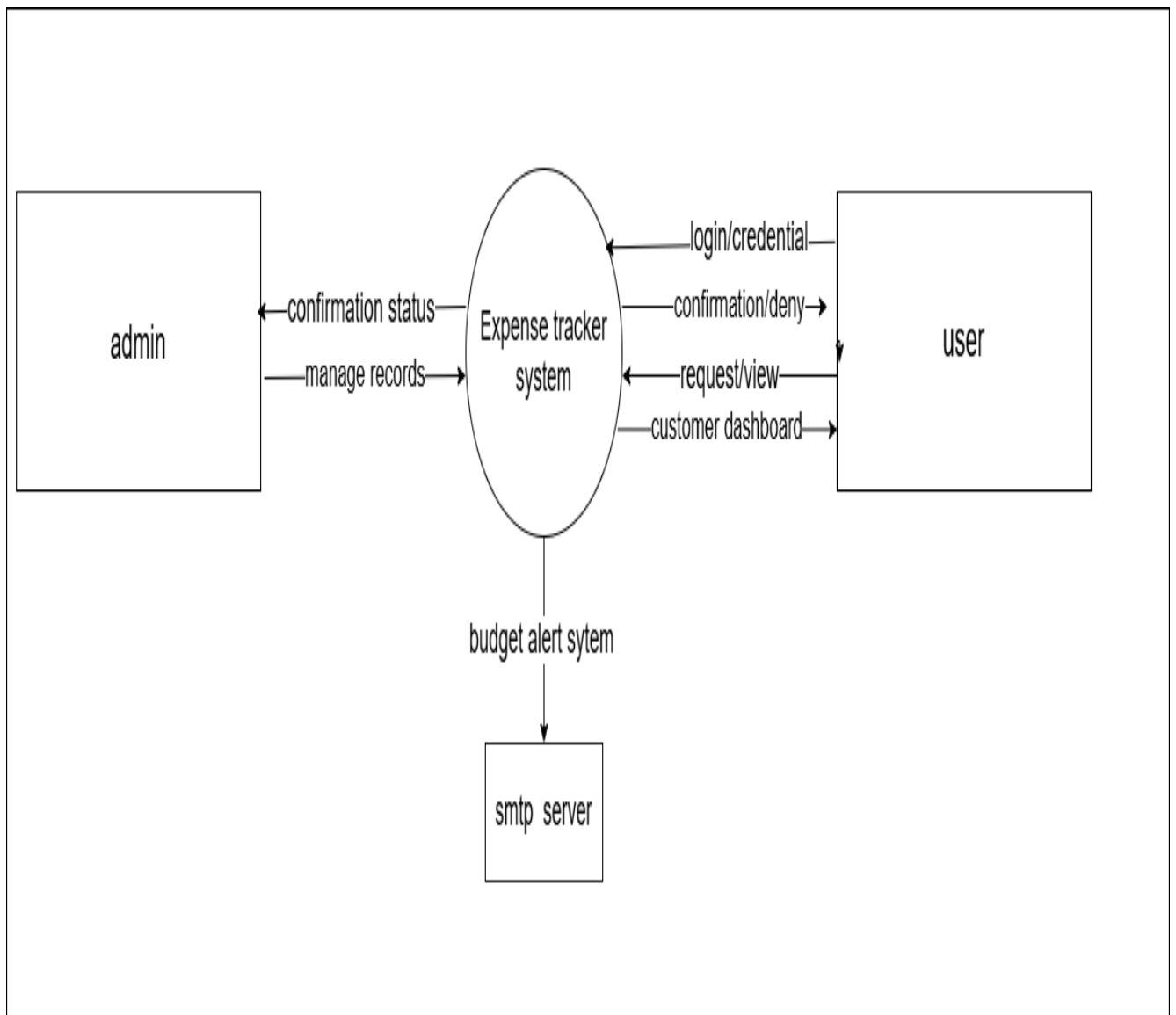
4.3 Generate CSV Report, 4.4 Export/Download Report

The DFD of the Expense Tracker system shows how data flows between users, admin, databases, and system processes.

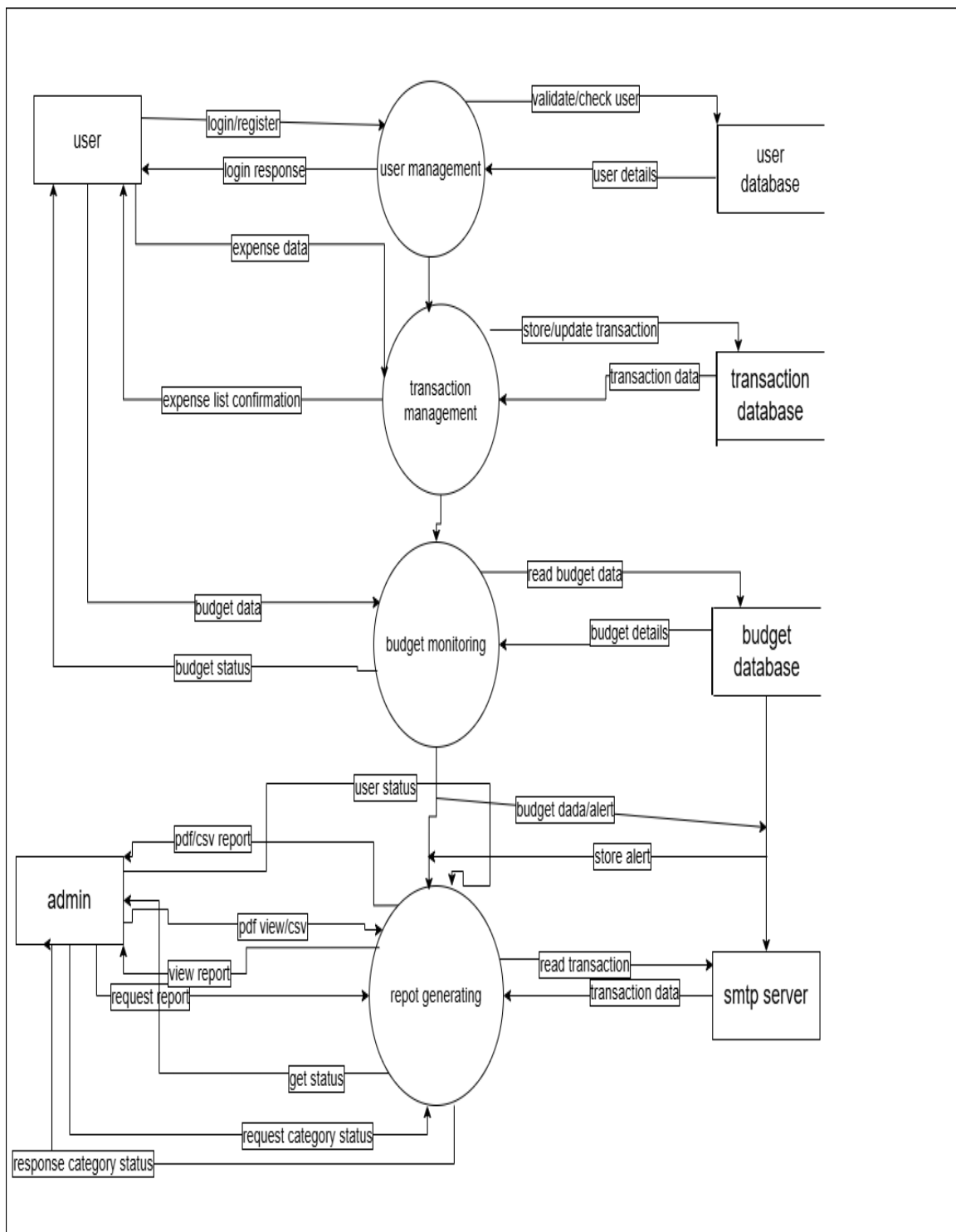
The main DFD components used are:

- External Entities (User, Admin, SMTP Server)
- Processes (User Management, Transaction Management, Budget Monitoring, Report Generation)
- Data Flows (Arrows showing movement of information)
- Data Stores (User, Transaction and Budget Databases)

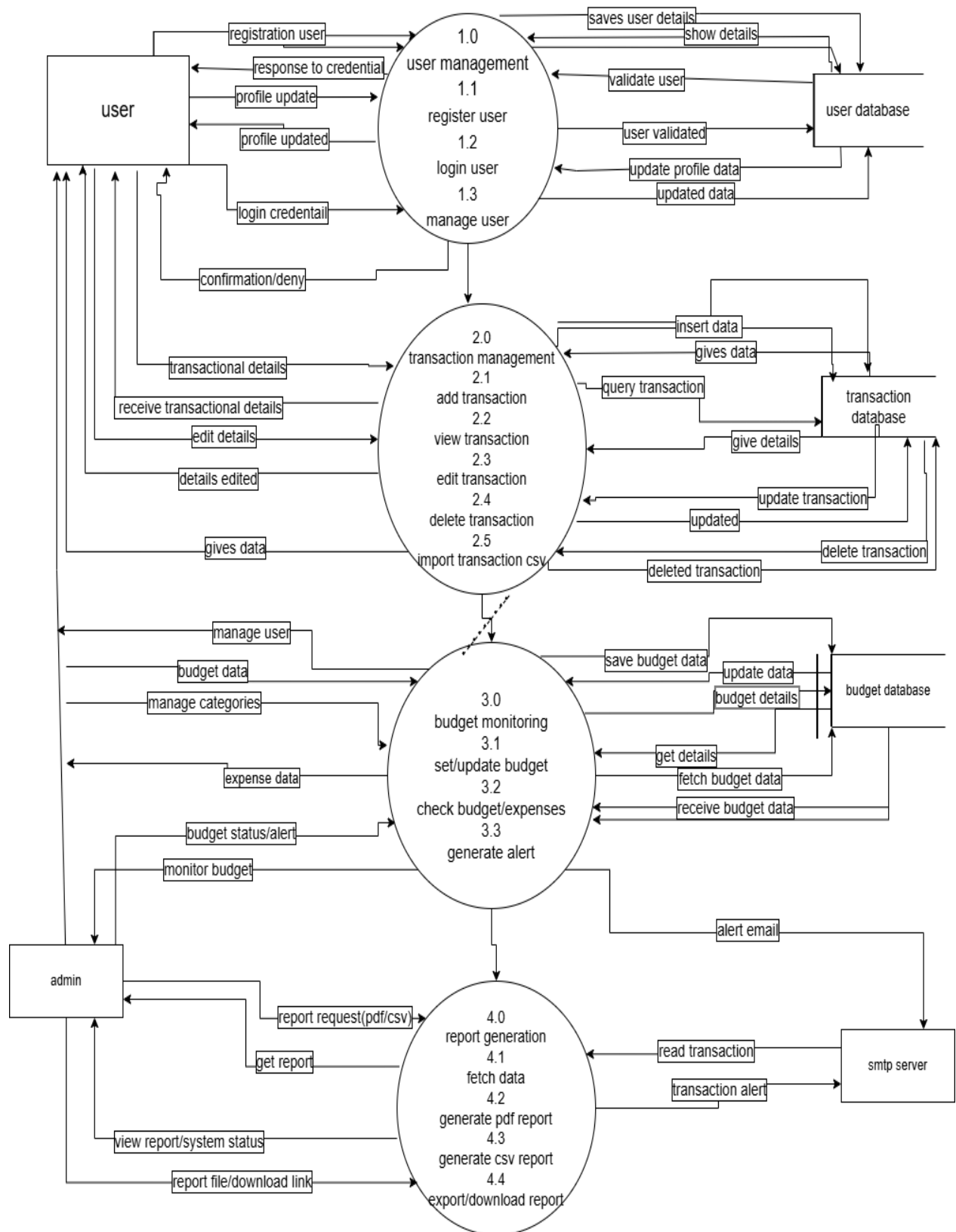
The Level 0 DFD provides an overall system view, Level 1 shows major processes and data stores, while Level 2 gives detailed breakdowns of internal sub-processes. Together, these diagrams explain the working logic of the entire system clearly and systematically



LEVEL 0



LEVEL 1



LEVEL 2

ER DIAGRAM

An ER diagram is a conceptual data modeling approach that visually represents the structure of a database. It shows how data is stored, organized, and related within a system. ER diagrams are widely used in database design because they provide a clear, high-level view of the data elements and their interconnections before actual implementation begins. The three major components of an ER diagram include entities, attributes, and relationships. Entities are real-world objects or concepts, such as a student, course, or employee. Each entity will have some attributes describing the characteristics of that entity. For instance, The **User** entity is the central entity of the system because all other data—categories, transactions, and budgets—belongs to a specific user. It stores account details and supports authentication, ownership, personalization, and security. **Purpose:** This entity stores all information related to users and acts as the parent entity in the database. Without the User entity: there is no ownership of transactions no user-specific categories no personalized budgets.

Category Entity: The Category entity organizes transactions into groups. Without categories, transactions are just numbers. **Purpose:** Categories organize transactions into groups. **Transaction:** This is the core entity. It stores every income and expense. **Purpose:** Stores money spent money received, dates, categories, descriptions Most system calculations depend on this entity. **Budget:** Budget stores spending limits. **Purpose:** Used for spending control, alerts, monitoring, utilization tracking

Keys in ER diagrams-which include primary keys and foreign keys-are another important part, ensuring that data are accurate and aiding in establishing relations between related tables. An early ER diagram reduces redundancy and logical errors, ensuring normalization. ER diagrams are very useful in software engineering, system analysis, and database development. They serve for planning and communicating the structure of data flow and system requirements and assist stakeholders in understanding those clearly. They are crucial in the design of efficient, reliable, and scalable databases

Components of ER Diagram

An ER diagram mainly consists of the following components:

- 1.Entities

2.Attributes

3. Relationships

4.Connectors and Symbol

TYPES OF ENTITIES

Entities is classified into two types:

Strong Entity:

A strong entity is an entity that:

- has its own primary key
- can exist independently
- does not depend on another entity for existence

It can be uniquely identified by its own attributes.

Ex: **Category** has foreign key user_ id, it still has its own unique identifier and can be treated as strong. Category: Primary Key = id

Weak Entity:

A weak entity:

- cannot exist independently
- depends on another entity
- often identified using parent entity

It usually lacks complete existence without its owner.

EX: **Budget**: can sometimes be viewed as weak.

Budget cannot exist without: User and Category.

Symbols used in entities:

Rectangle (): Represents an **Entity**.

Entities in Expense Tracker System:

1. User Entity: The User entity stores all information related to registered users and acts as the central controlling entity of the system.

2. Category Entity: Its function is to classify and organize transactions. Without category, With category.

3. Transaction Entity: This is the main operational entity. Its function is to store every income and expense.

4. Budget Entity: Stores spending limits. Its function is to: set limits ,track spending ,trigger alerts

Relationships

A relationship defines association between entities, It tells how one entity is connected to another.

Types of Relationships

- **One-to-One (1:1):**

One entity instance is associated with one instance of another entity.

Example: One user $\rightarrow$ one profile

- **One-to-Many (1:N):**

One entity instance is associated with many instances of another entity.

Example: One User has Many Categories

- **Many-to-Many (M:N):**

Many instances of one entity are associated with many instances of another.

Example: Many transactions are interconnected.

Symbols in Relationships

Diamond ($\diamond$): Represents a **Relationship**.

Example: One Category has Many Budgets, One Category has Many Transactions

Relationships in expense Tracker System

Listed below are the some relationships of expense Tracker System

Relationship 1 User $\rightarrow$ Category

One User has Many Categories

Example

Durgesh: Food, Travel, Bills

Functions: One user created many categories.

Relationship 2 User $\rightarrow$ Transaction

One User has Many Transactions

Example:

User: Durgesh

Transactions: 500 Food, 2000 Rent, 300 Salary

Function: Stores all user financial activity.

Relationship 3 User $\rightarrow$ Budget

One User has Many Budgets

Example:

Food Budget, Travel Budget

Function:

Allows multiple spending limits.

Relationship 4

Category $\rightarrow$ Transaction

One Category has Many Transactions

Example:

Food, Lunch, Groceries, Dinner

Function:

Classifies related transactions.

Attributes

attribute is a property or characteristic of an entity.

It describes what information is stored about that entity.

Example:

Entity: User

Attributes: username, email, password_hash

1. Attributes

Definition of Attribute

An attribute is a property or characteristic of an entity.

It describes what information is stored about that entity.

Example:

Entity: User

Attributes: username, email, password_hash

Types of Attributes Used in This System

(A) Simple Attributes

A simple attribute cannot be broken into smaller parts.

Examples in this system:

Username, email, amount, limit_amount

Example: Amount = 500

cannot be subdivided further.

(B) Composite Attributes

Composite attributes can be divided into smaller components.

Example in this system: date

can be divided into: day, month, year

Example: 24 / 04 / 2026

(C) Single-Valued Attributes

Store only one value for each record.

Examples: username, email, preferred_currency

One user has one email.

(D) Multi-Valued Attributes (Conceptual Use)

Can have multiple values.

Example: A user can have multiple categories:

Food, Travel, Bills

Although stored as separate records, conceptually this behaves like multi-valued data.

(E) Derived Attributes

Calculated from other stored values.

Examples: current_spent

derived from: all expense transactions.

Example: Food Transactions:

500, 700, 300

Current Spent = 1500

Derived.

(F) Key Attributes

Primary Key

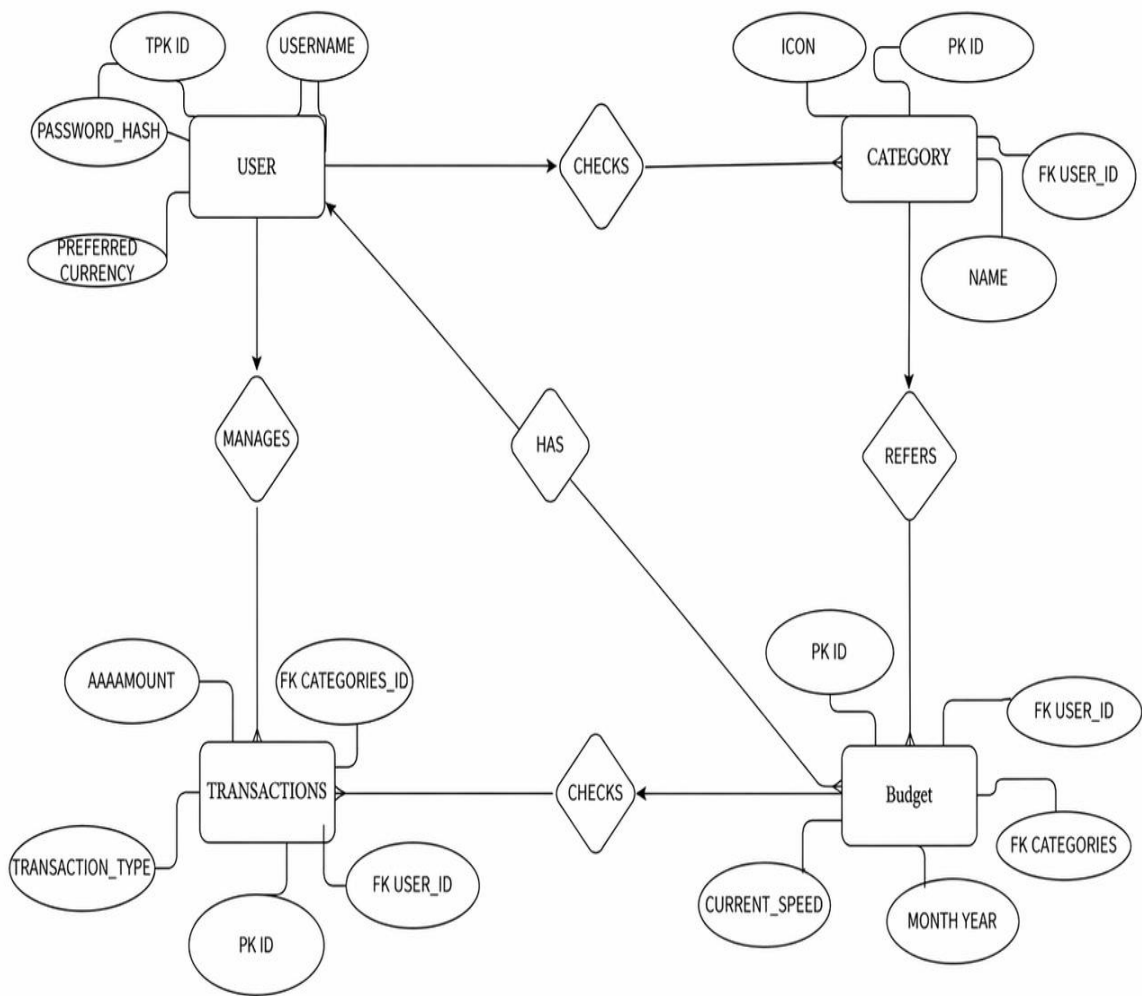
Uniquely identifies records.

Examples: User.id, Transaction.id

Foreign Key

Creates relationships.

Examples: Transaction.user_id, Transaction.category_id



ER DIAGRAM

USE CASE

A **Use Case Diagram** shows how a user interacts with a system.

In simple words: **Who uses the system, What the user can do in the system, How the user connects with those functions**

It shows the system from the user's point of view.

Example: User logs in, User adds expense, User views report

Components of Use Case Diagram

User: This is the person using the system. What user does

User can: Register, Login, Add expenses, Set budget, View reports

User uses all functions.

Functions: They show what system can do.

Example: Add Transaction

Meaning: User adds an expense.

Example: Food = 500 Stored in system.

LIKE this many example of functions are there.

Connections: Lines connect user to functions.

Example: User ----- Add Transaction

Meaning: User can add transaction.

Line simply means: User uses that feature.

System Box: Big outer box: This box contains all functions.

Example: Expense Tracker System

inside box: Login, Budget Dashboard

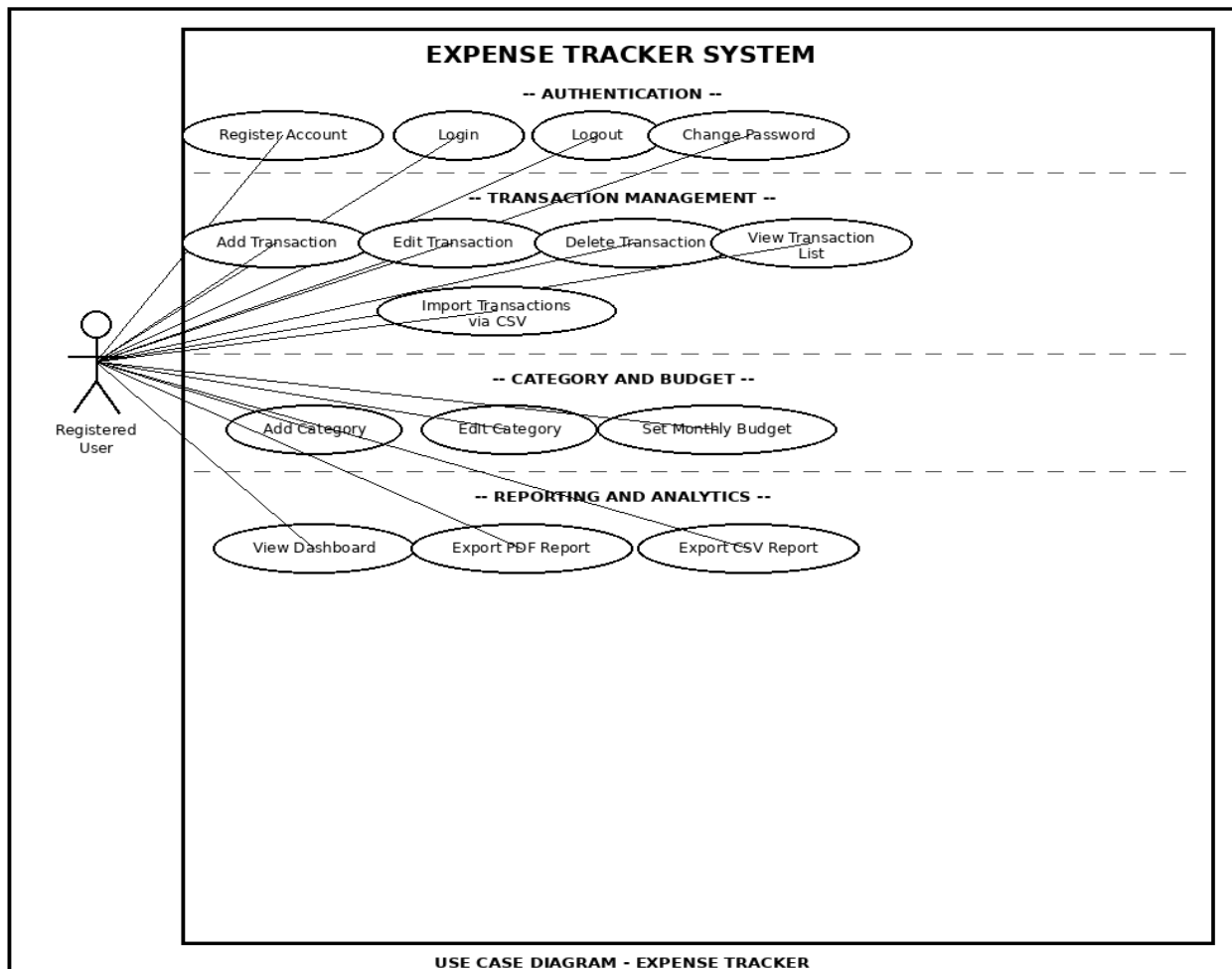
Outside box: User

Meaning: User uses system from outside. Functions are inside system.

Who uses system? : User

What can user do? Add expense, Set budget, View reporting->How? , Using connections,
Where? Inside system box

Use case diagram



Database Design

A **Database Design** shows how data is stored, organized, and connected in the system.

In simple words: What data is stored, Where it is stored, How tables connect,

How the system uses that data

Function of Database Design

The function of database design is to help the system:

- store user data
- store expenses and income
- organize records
- track budgets
- generate reports

Without database design, the system cannot manage data properly.

Simple Example

Suppose user enters: Food Expense = 500

Database does this: Stores user, Stores category (Food), Stores transaction (500), Checks budget

Updates dashboard

All this happens because of database design.

Components of Database Design

Your diagram has 4 main components:

Tables: A table stores one type of data.

Think of a table like an Excel sheet.

Each table has:

- Rows = records
- Columns = fields

Tables in Your Project

There are 4 tables.

Users Table: Stores user information.

Example: Durgesh

durgesh@gmail.com, INR

Function: Login account details, identifies owner of data

Categories Table

Stores expense groups.

Example: Food, Travel, Bills Function: Organizes transactions.

Instead of random numbers: 500,200

you know: 500 = Food, 200 = Travel

Transactions Table

Stores money records.

Example: Salary = 20000 Food = 500

Function: Stores: income, expenses dates, descriptions

This is the main table of system.

Budgets Table

Stores monthly limits.

Example: Food Budget = 5000

Function: Checks whether spending crosses limit.

Example: Spent 4000, 80% alert triggered

Columns: Columns store details inside tables.

Example: Users table columns: username, email, password

These describe user.

Transaction columns: amount date, description

Meaning: amount spent, when spent, why spent

Keys: Keys identify and connect data.

Primary Key (PK) Unique ID.

Example: User id =1, Transaction id = 25

Function: Every record gets unique identity.

Foreign Key (FK)

Links tables.

Example: transaction.user_id → users.id

Meaning: This transaction belongs to that user.

Another: transaction.category_id → category.id

Means: Expense belongs to Food category.

Relationships:

Relationships show how tables connect.

User → Categories

1 User → Many Categories

Example: One user has: Food, Bills, Travel

User → Transactions

1 User → Many Transactions

Example: One user may have 100 expenses.

User → Budgets

1 User → Many Budgets

Example: Food budget, Travel budget

Category → Transactions

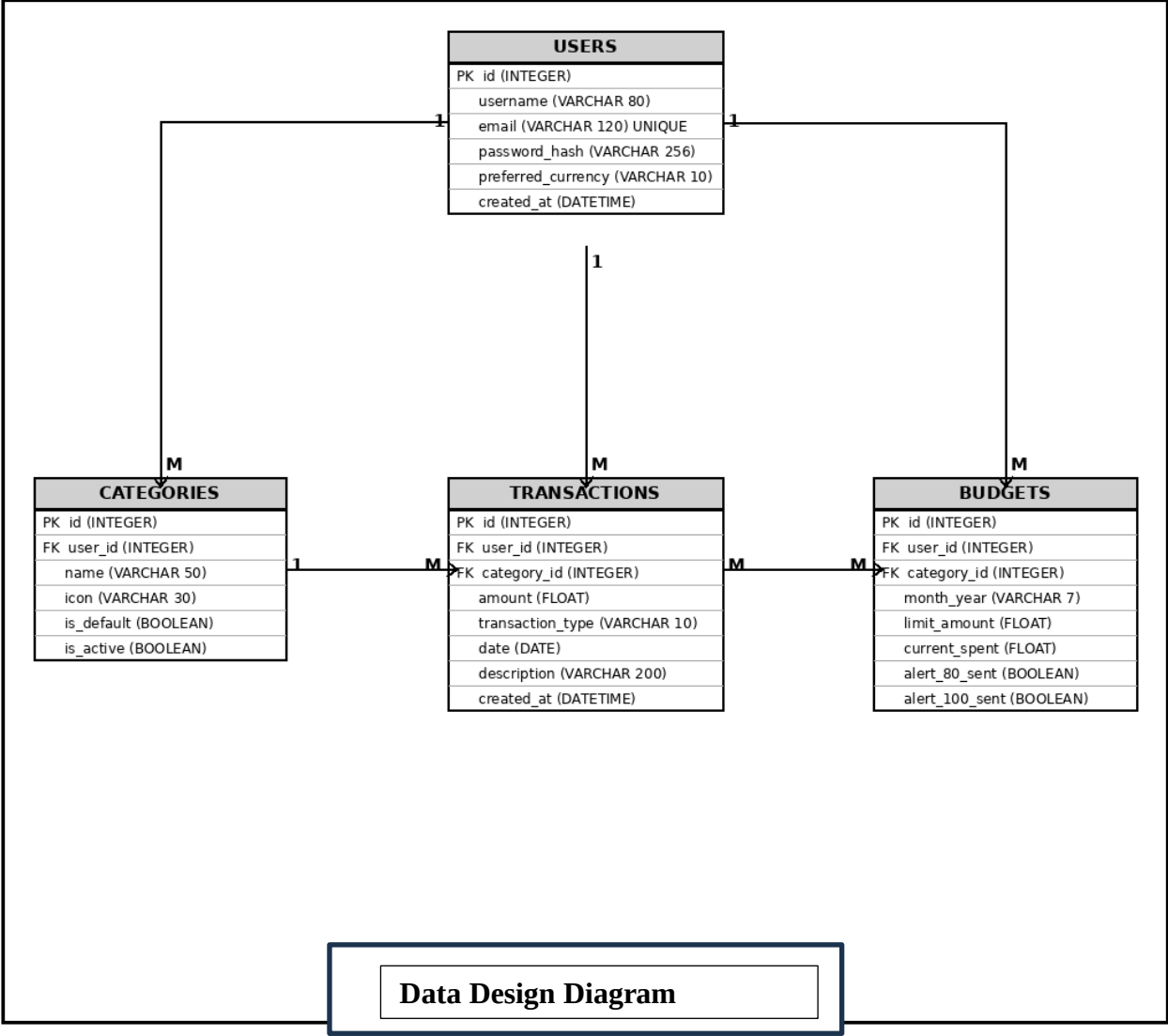
1 Category → Many Transactions

Example: Food category includes: Lunch Grocery Dinner

Category → Budgets

1 Category → Many Budgets

Example: Food-April, Food-May



INPUT AND OUTPUT DESIGN

What data goes into the system (**Input**), How the system processes that data (**Process**), What results the system shows (**Output**).

This is often called: Input → Process → Output (IPO)

Input Design

Input Design means designing **how data enters the system**.

It focuses on: what users enter, how data is collected, how wrong data is prevented, how data is validated before saving

Input Design = How the system takes information from the user.

Function of Input Design

Its purpose is to: collect correct data, avoid wrong entries, improve accuracy, keep data secure

Example:

If a user enters: Food Expense = 500

the system checks: Is amount entered?, Is 500 a valid number?, Is category selected?, Is date provided?

Components of Input Design

1. Registration Form

Used for creating account.

Inputs: Name, Email, Password, Currency

Example: Durgesh, durgesh@gmail.com, 12345, INR

Function: Stores user account details.

2. Login Form

Inputs: Email, Password

Function: Allows user to enter system.

3. Add Transaction Form

Inputs: Amount, Date, Category Income /Expense

DescriptionExample:500FoodExpense

Function: Stores money records.

4. Set Budget Form

Inputs: Category, Month Budget Limit

Example: FoodApril5000

Function: Sets spending limit.

5. CSV Import

Input: CSV file

Function: Imports multiple transactions at once.

Output Design

Output Design means designing **what results the system shows to user.**

Output Design = Information produced by the system

Function of Output Design

It helps user see: expenses, budgets, reports, summaries

Components of Output Design

1. Dashboard Output Shows summary.

Outputs: Total Income, Total Expense, Net Balance Budget, Progress Charts

Example: Income = 20,000Expense = 8,000Balance = 12,000.

Function: Gives quick financial overview.

2. Transaction List Output

Shows all transactions.

Example: Food 500Rent 3000Salary 20000

Function: View and filter records.

3. PDF Report Output

Creates downloadable report.

Contains: summary transaction table statistics

Function: Used for report printing.

4. CSV Export Output

Exports data to Excel.

Function: backup analysis, import elsewhere

5. System Messages

Shows feedback.

Examples: Transaction Added Successfully, Invalid Amount

Function: Tells user what happened.

What is Processing?

Processing is middle part.

Input → Processing → Output

System processing does: validates data, stores in database, updates budget, generates analytics
produces outputs

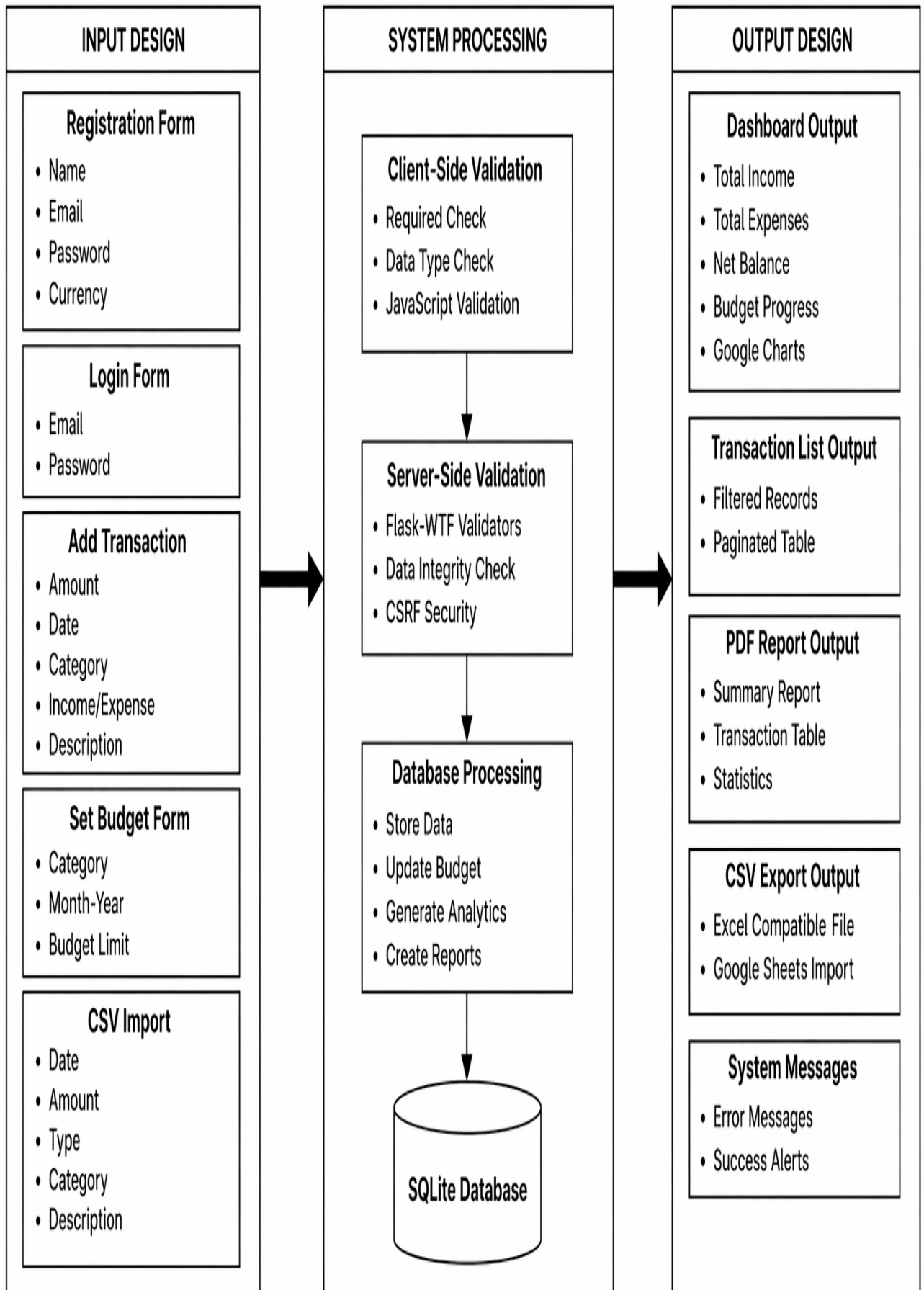
Real Example

Budget = 5000Current spent = 4500New expense = 700

Processing: $4500 + 700 = 5200$

Output: Budget exceeded alert shown, This is Input-Process-Output working.

INPUT - PROCESS - OUTPUT DESIGN



Implementation

IMPLEMENTATION

This chapter explains how the Expense Tracker system was actually developed and turned into a working application. In the previous chapter, the design of the system explained what features the project should have and how different parts of the system would interact. This chapter focuses on how those ideas were implemented using programming, database management, and supporting technologies.

In simple words, this chapter explains how the project was built, what tools were used, how each module works, and what processes run behind the scenes when a user performs activities such as registering an account, adding expenses, setting budgets, receiving alerts, viewing charts, or downloading reports.

The purpose of implementation is not only to make the system functional but also to make it secure, user-friendly, reliable, and efficient.

Tools and Technologies Used

Technology	Version	Purpose
Python	3.11+	Primary backend programming language
Flask	3.0.x	Web microframework — routing, views, session management
Flask-SQLAlchemy	3.1.x	ORM for database interaction
Flask-Login	0.6.x	User session and authentication management
Flask-WTF	1.2.x	Form validation and CSRF protection
Flask-Mail	0.10.x	SMTP email dispatch for budget alerts
Werkzeug	3.0.x	Password hashing (PBKDF2-SHA256) and utility functions
Report Lab	4.x	PDF report generation
SQLite3	Bundled	Local relational database engine
Bootstrap 5	5.3.x	Responsive frontend CSS framework (CDN)

Google Charts API	Current	Client-side interactive data visualisation (CDN)
HTML5 / CSS3	—	Frontend markup and styling
JavaScript (ES6+)	—	Frontend interactivity and async data fetching
Jinja2	3.1.x	Server-side HTML templating (bundled with Flask)
VS Code	Latest	Development IDE
Git	Latest	Version control

Module Description

The application is divided into separate modules. Each module performs a specific task. Breaking the project into modules makes development easier and the system more organized.

User Authentication Module

This module controls user access.

Its purpose is to ensure only authorized users can use the system.

Main functions: Registration Login, Logout, Password Security

User Registration

When a new user creates an account:

User enters: Name, Email, Password

The system performs checks:

- Is email valid?
- Is password strong enough?
- Does account already exist?

If all checks pass: Account is created. Password is encrypted before saving.

Example:

Input: Name: Amit Password: amit123

Stored: Password becomes secure hash, not plain text.

User Login

Steps: User enters credentials. System checks database.

1. If correct: Access granted.
4. If incorrect: Login denied.

Example: Wrong password: Error shown: "Invalid credentials"

Logout

When user clicks Logout: Session ends, User must log in again, This improves security.

Transaction Management Module

This is one of the most important modules.

It handles: Adding income, Adding expenses, Editing transactions, Deleting records, Viewing history

Adding Expenses

Suppose user spends: ₹250 on Transport

User enters: Date, Category, Amount, Description

System stores it, After saving: Expense list updates, Budget calculation updates, Charts update

Editing Transaction

If amount entered wrong: ₹2500 instead of ₹250

User can edit and correct it.

Deleting Transaction

Wrong entries can be removed.

Before deleting: System verifies ownership, Only owner can delete their own data.

Transaction Filtering

Users can search by: Date, Category, Expense type

Example: Show only Travel expenses for March.

CSV Import: Bulk records can be uploaded.

Example: Date, Category, Amount

2026-03-01,Food,500

2026-03-02,Travel,300

System checks: Correct format Positive amount, Valid categories

Then imports records.

Budget Management Module

This module helps users control spending.

Users define spending limits.

Example: Food Budget = ₹5000

Alert System Module

This module sends automated notifications.

Two alert levels:

80% Alert: Warning email: "You have used 80% of your Food budget."

100% Alert: Critical email: "Your Food budget has been exceeded."

Duplicate Alert Prevention

If warning already sent:

System does not send it repeatedly.

This avoids unnecessary spam.

Analytics Dashboard Module

This module analyzes data and shows charts.

It helps users understand spending patterns.

Category Analysis

Example: Food = 3000, Travel = 2000
Shopping = 1000

Displayed as pie chart.

Monthly Trend

January = 7000, February = 8000, March = 6500

Shows spending pattern over time.

Weekly Breakdown

Monday = 300, Friday = 1500

Shows which days expenses rise.

Report Generation Module

This module creates reports.

Types: PDF reports, CSV exports

Report Includes: Total income, Total expenses ,Balance, Transactions list

Example: Income = 20,000, Expenses = 15,000

Balance: $20000 - 15000 = 5000$

Balance = ₹5000

Development Environment

The development environment refers to the software, tools, and system setup used to build, test, and run the Expense Tracker application during development. In simple terms, it is the complete workspace where the project was created, coded, debugged, and tested before becoming a working application.

A proper development environment is important because it helps developers write code efficiently, test features correctly, identify errors quickly, and ensure the application runs smoothly. For this project, a simple and practical development setup was used so that the system could be developed and managed easily.

operating System

The project was developed on **Windows 11**.

Windows 11 was used because it provides a stable platform for Python development and supports all tools required for the project, such as Visual Studio Code, Git, Flask, and SQLite.

The operating system was used for: Managing project files and folders Running Python and Flask commands Installing required libraries, Testing the application in a web browser, Managing the database file

Windows command prompt and terminal were used frequently for running the application and installing dependencies.

Example commands used: `python app.py`

A **Python** virtual environment (venv) was used to isolate project dependencies from the system

Python installation. All dependencies are listed in a requirements.txt file for reproducibility.

All dependencies are listed in a requirements.txt file for reproducibility.

The **database** was initialised using Flask-SQLAlchemy's `db.create_all()` method on first run, which automatically creates all tables from the SQLAlchemy model definitions For email testing during development, Mailtrap.io was used as a sandboxed **SMTP** relay that captures outgoing emails without delivering them, making it possible to test the budget alert system without sending real emails.

Integrated Development Environment (IDE)

Visual Studio Code (VS Code) was used as the primary code editor.

VS Code was chosen because it is lightweight, easy to use, and provides many helpful features for coding.

It was used for: Writing Python code, Creating HTML templates, Styling pages with CSS, Writing JavaScript for frontend features, Debugging errors , Managing project folders.

Useful VS Code features used during development included: Syntax highlighting, Auto-completion (IntelliSense), Code suggestions, Error detection, Integrated terminal

Debugging support. These features made coding faster and helped reduce mistakes.

For example, if a bracket was missing or a function had an error, VS Code could highlight the problem immediately.

Python Environment

The application was developed using **Python 3.x**.

Python was the main programming language used to implement backend logic.

A **virtual environment (venv)** was created to isolate project dependencies from the system Python installation.

This is important because it keeps project libraries separate and avoids conflicts with other Python projects.

Virtual environment creation:

```
python -m venv venv
```

Activation command: `venv\Scripts\activate`

After activation, required packages were installed.

Example: `pip install flask`

Other project libraries such as Flask-SQLAlchemy, Flask-Login, and Flask-Mail were installed similarly.

All dependencies were stored in: `requirements.txt`

This makes the project reproducible on another machine.

Flask Development Server

During development, the project was run using Flask's built-in development server.

Command used: `flask run --debug`

The Flask server was used for: Running the application locally, Testing pages in browser, Processing requests, Debugging application errors

The application typically ran at: `http://127.0.0.1:5000`

This is also called localhost.

When the developer opened this address in a browser, the Expense Tracker application loaded.

Database Environment

The system used **SQLite3** as the development database.

SQLite stores data in a single file, usually: `expenses.db`

This database stored: User information, Transactions, Budgets, Categories, Alert records

The database was initialized using:

```
db.create_all()
```

This command automatically created required tables based on model definitions.

Frontend Testing Environment

The user interface was tested in **Google Chrome**.

Chrome was used to test: Form behavior, Dashboard layout, Bootstrap responsiveness,

Chart rendering, JavaScript interactions

Developer Tools in Chrome were also used.

They helped inspect: HTML structure, CSS styling, JavaScript errors, Network requests

For example, if a chart failed to load, browser console errors could help identify the issue.

Email Testing Environment

Since the project includes automated budget alert emails, email functionality also needed testing.

During development, SMTP configuration was tested using a development email account.

This was used to verify: Warning alert emails, Budget exceeded emails

Email formatting, Successful mail delivery

Example: If spending reached 80% of a budget, the developer checked whether the warning email was triggered correctly.

This ensured the alert system worked before deployment.

Testing

TESTING

Testing is a very important step in any project because it helps us understand whether the system is actually working as expected. Simply building the application is not enough — we also need to check if every feature behaves correctly in real situations.

In this project, testing was done not only to confirm that the system works with correct inputs, but also to see how it behaves when users make mistakes. In real life, users may enter wrong data, leave fields empty, or try unexpected actions. So, the system must be able to handle such situations properly without crashing.

The main goal of testing in this project was to ensure that all features — such as user registration, login, adding transactions, setting budgets, receiving alerts, and generating reports — work smoothly and give correct results every time. Special attention was given to checking that errors are handled properly and clear messages are shown to the user.

Different types of testing methods were used to check the system from multiple angles. Each part of the application was tested step by step, and then the complete system was tested as a whole. This helped in identifying even small issues and fixing them before final completion.

Overall, testing helped improve the reliability, accuracy, and security of the system. It ensured that the application is stable, user-friendly, and ready for practical use.

Testing Methods

In this project, different types of testing were used to make sure the Expense Tracker system works properly in real-life situations. Testing was not only done to check correct results, but also to see how the system behaves when users make mistakes or enter wrong data. This is important because in real usage, users may not always follow the correct steps.

By using multiple testing methods, the system was checked from different angles. This helped in improving the accuracy, stability, and overall performance of the application. As a result, the system became more reliable and user-friendly.

Testing methods used in this Expense tracker system Unit Testing, Integration Testing, Functional Testing (Black-Box Testing), Boundary Value Testing, Negative Testing Security Testing

Unit Testing

Unit testing is used to test small parts of the system separately. In simple terms, each function or feature is checked one by one to make sure it works correctly.

In this project, unit testing was done for important parts such as password security, budget calculations, transaction validation, and CSV file handling. For example, password functions were tested to ensure that passwords are safely stored and correctly verified during login.

This type of testing helped make sure that each individual part of the system is working properly before combining everything together.

Integration Testing

Integration testing is used to check whether different parts of the system work well together.

In this project, complete workflows were tested, such as registering a user, logging in, and accessing the dashboard. Another example is adding a transaction and checking whether the

dashboard updates automatically. Budget alerts were also tested to ensure they trigger correctly when spending crosses the limit.

This testing made sure that all parts of the system are properly connected and work smoothly as a complete application.

Functional Testing (Black-Box Testing)

Functional testing checks whether the system works according to user requirements.

In this method, the system is treated like a “black box,” meaning only the input and output are checked, not the internal code.

In this project, all main features were tested manually using a web browser. This included user registration, login, transaction management, budget setting, viewing charts, and exporting reports.

This testing helped confirm that the system behaves correctly from a user’s point of view.

Boundary Value Testing

Boundary value testing focuses on testing values at the limits where errors are most likely to occur.

In this project, different edge cases were tested, such as entering zero amount, very large values, negative numbers, empty text fields, and maximum-length inputs. Budget alerts were also tested exactly at 80% and 100% usage levels.

This helped ensure that the system handles extreme and edge values properly without causing errors.

Negative Testing

Negative testing involves giving wrong or invalid inputs to check how the system reacts.

The goal is to ensure that the system does not crash and instead shows proper error messages.

In this project, tests included leaving fields empty, entering text in number fields, using duplicate email addresses, and entering incorrect login details.

The system handled all these cases correctly by showing clear error messages and preventing invalid data from being saved.

This made the system more strong and dependable.

Security Testing

Security testing was done to make sure that user data is safe and protected.

Important checks included submitting forms without a valid security token (CSRF), trying to access another user's data, verifying password storage, and checking session logout behavior.

The results showed that the system blocks unauthorized access, protects user data, and keeps passwords secure using encryption.

Test Cases

Test cases are used to check whether each feature of the system is working correctly or not. In simple terms, a test case is a situation where we give some input to the system and check if the output is as expected.

In this project, different test cases were created to verify all major functionalities such as user registration, login, transaction management, budget alerts, report generation, and security features.

Each test case includes the input given, the expected result, and the actual outcome.

All the following test cases were executed, and the system performed as expected.

Test Case Table

TC No.	Test Scenario	Input	Expected Output	Status
TC01	User Registration (Valid)	New email, strong password	Account created successfully	PASS
TC02	Duplicate Registration	Existing email	Error message displayed	PASS
TC03	Login (Valid)	Correct email & password	Dashboard opened	PASS
TC04	Login (Wrong Password)	Incorrect password	Error message shown	PASS
TC05	Add Transaction	Amount: 500	Transaction successfully saved	PASS
TC06	Invalid Amount	Amount: 0	Validation error displayed	PASS

TC No.	Test Scenario	Input	Expected Output	Status
TC07	Empty Field	Date not entered	Error message shown	PASS
TC08	Edit Transaction	Update amount	Updated data displayed	PASS
TC09	Delete Transaction	Delete selected record	Record removed successfully	PASS
TC10	Unauthorized Access	Access another user's data	Access denied	PASS
TC11	Set Budget	Limit = 3000	Budget saved successfully	PASS
TC12	Budget 80% Alert	Spend 2400	Warning email sent	PASS
TC13	Budget 100% Alert	Spend 3000	Alert email sent	PASS
TC14	Dashboard Charts	Open dashboard	Charts displayed correctly	PASS
TC15	Export PDF	Select month	PDF file downloaded	PASS
TC16	Export CSV	Select month	CSV file downloaded	PASS
TC17	CSV Import (Valid)	Upload correct file	Data imported successfully	PASS
TC18	CSV Import (Invalid)	Missing column in file	Error message shown	PASS
TC19	CSRF Protection	Submit form without token	Request rejected	PASS
TC20	Mobile View	Open on small screen	Layout adjusts properly	PASS

Results of Testing

After running all the test cases, it was observed that the system is working properly and meeting the expected requirements. Each feature of the application was tested carefully, and the results showed that the system behaves correctly in different situations.

All the test cases were executed successfully, and the outputs matched the expected results. No major errors or system failures were found during testing. This indicates that the application is stable and ready for use.

However, during testing, a few small issues were identified. These were minor problems and were fixed before finalizing the project.

Issues Found and Solutions

Issue

1:

The system was allowing users to enter an amount value of 0 while adding a transaction. This is not practical because a transaction should always have a value greater than zero.

Solution:

A validation rule was added to ensure that the amount must be greater than 0. Now, if a user enters 0, the system shows an error message and does not allow the transaction to be saved.

Issue

2:

During CSV import, the system was accepting negative values for transaction amounts, which could lead to incorrect data.

Solution:

A validation check was added to reject any negative values during CSV import. Now, only valid positive amounts are accepted, and incorrect rows are flagged.

Security Results

Security testing was also performed to make sure that user data is protected.

The results confirmed that:

- CSRF protection is working properly
- Users cannot access other users' data
- Passwords are stored securely and are not visible
- Unauthorized access is blocked by the system

These checks ensure that the application is safe and reliable for users.

Final Testing Result

Overall, the system performed well in all areas. The testing process confirmed that:

- All features are working correctly
- Errors are handled properly with clear messages
- Data is stored and processed securely
- The system provides a smooth and easy user experience
- The application works well across different browsers and devices

In conclusion, the testing phase proved that the Expense Tracker system is reliable, secure, and ready for practical use.

Results and Discussion

System Output

System output means the results generated by the application when a user performs actions in the system. Whenever a user enters data, clicks a button, or requests a report, the system processes that input and produces an output.

User Registration and Login Output

The first output produced by the system is related to user authentication.

When a new user creates an account, the system saves the user details such as name, email, and password in the database. For security, the password is stored in encrypted form instead of plain text.

After successful registration, the system displays a success message and sends the user to the login page.

When the user enters correct login details, the system verifies the credentials and opens the user dashboard.

After login, the system shows: User dashboard, Financial summary information

Navigation menu, Active user session

This means the user can start using the application immediately.

If wrong credentials are entered, the system displays proper error messages such as: Invalid email or password, Required fields missing, Login failed

This helps prevent errors and improves security.

Output Generated: Registration success confirmation, Login success output, Dashboard display after login, Error messages for invalid attempts

Transaction Management Output

The main function of the system is recording and managing transactions.

When a user adds an income or expense, the system stores information such as: Date, Category, Description, Amount, Transaction type

After saving the transaction, the system immediately updates all calculations automatically.

Updated outputs include: Total income, Total expenses, Net balance, Category, totals, Dashboard values.

For example:

If a user enters: Salary = ₹25,000 Food Expense = ₹2,000

The system automatically calculates: Income = ₹25,000, Expense = ₹2,000, Balance = ₹23,000

The transaction also appears in the transaction list.

The system also allows users to: Edit transactions, Delete records, Search transactions,

Apply filters, View paginated results

This makes transaction handling flexible and easy.

Output Generated: Updated transaction, records Balance calculations, Filtered results, Edited records, Deleted transaction updates.

Budget Monitoring Output

The budget module helps users control their spending.

Users can set monthly budgets for categories like: Food, Transport, Shopping, Bills

Example: Food Budget = ₹5,000

If spending reaches: ₹3,500 → 70% used, ₹4,000 → 80% used (warning alert), ₹5,000 → 100% used (limit exceeded alert)

The system displays this information through: Progress bars, Spending percentages, Warning notifications, Alert emails

Output Generated: Budget progress output, Warning alerts, Budget limit notifications, Visual progress indicators

Email Alert Output

The system also generates automatic email alerts.

When a user reaches 80% of a budget, a warning email is sent.

Example: "Warning: You have used 80% of your Food budget.", When the user reaches 100%, another alert is generated.

Example: "Alert: Your Food budget has been fully utilized."

Each alert is sent only once in a budget period so the user does not receive repeated messages.

Email output includes: Category name, Current spending, Budget limit, Alert type, Dashboard link

Output Generated: Budget warning email, Budget limit exceeded email, Spending notification messages

Analytics Dashboard Output

The dashboard produces the main visual outputs of the system. After login, the user sees a dashboard containing:

Summary Cards

Three cards display: Total Income, Total Expense, Net Balance

These values update automatically whenever a new transaction is added.

Budget Progress Bars

Each category shows: Spending percentage, Remaining budget, Progress status

Charts and Graphs

The system displays three charts.

Category Pie Chart

Shows expense distribution by category.

Example: Food – 35%, Transport – 20%, Shopping – 25%, Bills – 20%

Monthly Trend Chart

Shows how spending changes over different months.

This helps users identify patterns.

Weekly Spending Chart

Shows short-term weekly expense behavior.

The charts are interactive.

When the user moves the cursor over data, exact values appear.

Output Generated: Financial summaries, Interactive charts, Spending analysis, Budget visualizations

Report Export Output

The system allows users to export reports.

Reports can be downloaded in: PDF format, CSV format

PDF Report Output

Generated reports include: User details, Report period, Income summary, Expense summary, Transaction records

These reports can be viewed or printed.

CSV Export Output

CSV files contain: Column headers, Transaction rows, Spreadsheet-ready format

These can be opened directly in: Microsoft Excel, Google Sheets

without formatting problems.

Output Generated: PDF reports, CSV exports, Printable financial summaries

Performance Analysis

Performance analysis shows how fast, accurate, and reliable the system is.

Testing was done under normal single-user conditions.

Speed Performance

The average response times observed were:

Operation	Time
-----------	------

Dashboard Load	380 m/s
----------------	---------

Chart Load	120 m/s
------------	---------

Transaction Page	500 m/s
------------------	---------

PDF Report	800 m/s
------------	---------

CSV Export	Under 100 ms
------------	--------------

These results show the system responds very quickly.

Users experience almost no delay.

Database Performance

SQLite3 database performed efficiently.

Testing showed: Fast data storage, Quick record retrieval, Fast query execution, Small database size

Even with around 3,650 records, the database remained below 5 MB.

This shows the database is suitable for personal finance management

Accuracy Performance

Calculations were checked manually.

The following were verified: Income totals, Expense totals, Net balance

Budget percentages, Report values

Example:

Income = ₹10,000

Expense = ₹4,000

Balance: ₹6,000

All calculations were accurate.

Memory Usage

The application used low system resources.

Approximate memory usage: 50–80 MB RAM

This is very low and can run on ordinary systems.

User Interface Description

The user interface is the visible part of the system where users interact.

The interface was designed to be simple, clean, and easy for beginners.

It is built using Bootstrap 5 and supports: Desktop, Tablet, Mobile devices.

The layout adjusts automatically according to screen size.

Home Page

The home page includes: Project title, Navigation menu, Login option, Register option.

Introduction section, It guides users into the system.

Login and Register Page

These pages contain simple forms.

Register form includes: Name, Email, Password, Confirm password.

Login form includes: Email, Password

Validation messages help users avoid errors.

Dashboard Page

The dashboard contains:

Summary Cards

Displays: Total Income, Total Expense, Net Balance

Budget Section

Shows: Budget limits, Spending percentages, Progress bars

Charts Section

Includes: Pie Chart, Trend Chart, Weekly Chart

Charts stack vertically on mobile and appear side by side on desktop.

Add Transaction Form

The transaction form contains: Date, Category, Amount Type, Description

It also provides: Input validation, Error messages, User-friendly labels

This reduces data entry mistakes.

Transaction List Page

This page shows transactions in table form.

Columns include: Date, Description, Category, Type, Amount

Features include: Search, Filters, Pagination, Edit option, Delete option

This makes records easy to manage.

Budget Management Page

This page allows users to: Set budgets, Edit limits, Track spending, View warnings

Progress indicators make monitoring easy.

Report Export Page

Users select: Month, Year, Export type (PDF or CSV)

The **Interface uses:** White background, Dark navigation sidebar, Blue buttons, Green income indicators, Red expense indicators, Orange warning alerts Text sizes are consistent for readability.

Conclusion and Future Scope

Conclusion

Conclusion

This project, ***Expense Tracker System***, was created to solve a simple but very common problem — managing and understanding daily expenses. In today's digital lifestyle, people spend money in many different ways, but often do not keep proper track of it. Because of this, it becomes difficult to understand spending habits or control unnecessary expenses. This system was designed to make that process easier, clearer, and more organized.

After completing and testing the project, it is clear that the system successfully meets its main goals. A user can quickly register and start using the application without any confusion. Adding income and expenses is simple, and all records are organized neatly into categories. The dashboard instantly shows updated results, which helps users clearly see where their money is going without needing any technical or financial knowledge.

One of the strongest features of this system is budget tracking. Users can set limits for different categories, and the system automatically warns them before they overspend. This small feature can make a big difference in helping users develop better financial habits. Another important point is privacy — all user data stays secure and is not shared with any external party, which makes the system trustworthy.

The application also provides report generation in PDF and CSV formats. This allows users to download and review their financial data anytime, which is useful for planning and analysis. These features make the system not only functional but also practical for everyday use.

From a learning perspective, this project was very helpful. It gave hands-on experience in building a complete web application using technologies like Python, Flask, SQLite3, Bootstrap, and Google Charts. More importantly, it showed how different parts of a system work together. It also helped improve skills like problem-solving, debugging errors, and testing the application properly.

Even though the system works well, there are a few limitations. For example, SQLite3 is not ideal for handling many users at the same time, and manual data entry can take extra effort from users. However, these limitations are understandable for a student-level project and do not reduce the usefulness of the application.

In the end, the Expense Tracker system can be considered a successful project. It is simple to use, works efficiently, and helps solve a real-life problem. It also shows a good understanding of technical concepts and practical implementation.

FUTURE SCOPE

Future Scope

Although the current system works well and meets its main objectives, there are many ways it can be improved in the future to make it more powerful, convenient, and user-friendly.

One important improvement would be **bank and UPI integration**. At present, users need to enter their transactions manually, which can take time. If the system is connected to bank accounts or UPI services, it can automatically fetch transaction data. This will save time and make expense tracking more accurate and real-time.

Another useful enhancement is converting the system into a **mobile application (Android/iOS)**. Most users prefer using mobile apps for daily activities. A mobile version would allow users to easily track their expenses anytime and anywhere, making the system more practical for everyday use.

The system can also be improved by adding **multi-currency support**. This feature would be helpful for users who deal with international payments. By using live exchange rates, the system can calculate and display accurate values in different currencies.

In the future, **AI-based insights** can also be added. The system could study the user's spending patterns and provide smart suggestions. For example, it could warn users about unusual spending, predict future expenses, or suggest better budgeting strategies. This would make the system more intelligent and helpful.

Another enhancement could be **multi-user or family account support**. This would allow multiple users to manage shared expenses together, such as in a family or group. It would make the application more useful for managing household budgets.

Adding **cloud storage and synchronization** is also a valuable improvement. This would allow users to access their data from different devices and ensure that their data is safely backed up. It would also prevent data loss.

The reporting feature can be improved by allowing **custom date range reports**. Instead of only monthly reports, users could generate reports for any time period, such as weekly, quarterly, or yearly. This would provide more flexibility for analysis.

Finally, adding an **offline mode** would make the application more reliable. Users would be able to use the system even without an internet connection, and the data could be synchronized later when the connection is available.

Source Code

Code Structure

This chapter presents the key code snippets that form the core of the application. These excerpts illustrate the most important technical decisions made during implementation. The complete source code is available in the project submission folder.

Code Structure

The application follows Flask's recommended project layout using the Application Factory pattern with Blueprints for modular organization. The directory structure is as follows:

expense_tracker/

```
|— app.py          # Application factory (create_app function)
|— requirements.txt # Python package dependencies
|— templates/
|   |— base.html      # Base layout with sidebar navigation
|   |— dashboard.html # Main dashboard with charts
|   |— expense_form.html # Transaction list, add, edit templates
|   |— expense.html
|   |— budgets.html    # Budget management templates
|   |— budget_form.html # Login, register templates
|   |— login.html
|   |— requirement.html
|   |— confirm_delete.html
|   |— admin_categories.html
```

```

|   └─admin_dashboard.html

|   └─admin_expenses.html

|   └─admin_user.html

└─ instances

└─ expenses.db    # database module

```

Important Code Snippets

User model with password hashing

Extends UserMixin for Flask-Login. Hashes passwords with Werkzeug. First registered user auto-becomes admin.

```

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    email = db.Column(db.String(254), unique=True, nullable=False)
    password_hash = db.Column(db.String(256), nullable=False)
    preferred_currency = db.Column(db.String(10), default='INR')
    date_joined = db.Column(db.DateTime, default=db.func.now())
    is_admin = db.Column(db.Boolean, default=False)

    expenses = db.relationship('Expense', backref='user',
                              cascade='all, delete-orphan')

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

```

Budget model with computed properties

Calculates live spending vs. limit with get_spent() via an aggregate SQL query, and percentage for progress bars.

```

class Budget(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    category_id = db.Column(db.Integer, db.ForeignKey('category.id'), nullable=False)
    monthly_limit = db.Column(db.Numeric(10, 2), nullable=False)
    month = db.Column(db.Date, nullable=False) # always 1st of month

```

```

def get_spent(self):
    result = db.session.query(func.sum(Expense.amount)).filter(
        Expense.user_id == self.user_id,
        Expense.category_id == self.category_id,
        db.extract('year', Expense.date) == self.month.year,
        db.extract('month', Expense.date) == self.month.month
    ).scalar()
    return result or Decimal('0')

def get_percentage(self):
    if self.monthly_limit > 0:
        return min(int((self.get_spent() / self.monthly_limit) * 100), 100)
    return 0

```

DB init + category seeding

Seeds default categories on first run using an idempotent check — safe to run every startup.

```

DEFAULT_CATEGORIES = [
    ('Food & Dining', 'fa-utensils', '#e74c3c'),
    ('Rent & Housing', 'fa-home', '#3498db'),
    ('Transportation', 'fa-car', '#f39c12'),
    # ...more categories
]

def seed_categories():
    for name, icon, color in DEFAULT_CATEGORIES:
        if not Category.query.filter_by(name=name).first():
            db.session.add(Category(name=name, icon=icon, color_hex=color))
    db.session.commit()

with app.app_context():
    db.create_all()
    seed_categories()

```

Budget Alert Trigger Function

```

from flask_mail import Message

from . import mail

```

```

def check_and_send_budget_alert(user, category, month_year):

    budget = Budget.query.filter_by(

```

```
        user_id=user.id, category_id=category.id, month_year=month_year
    ).first()

    if not budget:

        return

    total_spent = db.session.query(db.func.sum(Transaction.amount)).filter(

        Transaction.user_id == user.id,

        Transaction.category_id == category.id,

        Transaction.transaction_type == 'expense',

        db.func.strftime('%Y-%m', Transaction.date) == month_year

    ).scalar() or 0.0

    utilisation = (total_spent / budget.limit_amount) * 100

    if utilisation >= 80 and not budget.alert_80_sent:

        send_alert_email(user.email, category.name, total_spent,

                        budget.limit_amount, 80)

        budget.alert_80_sent = True

    if utilisation >= 100 and not budget.alert_100_sent:

        send_alert_email(user.email, category.name, total_spent,

                        budget.limit_amount, 100)

        budget.alert_100_sent = True

    db.session.commit()
```


8.2.3 Analytics API Endpoint for Category Pie Chart

```
from flask import Blueprint, jsonify

from flask_login import login_required, current_user

from .models import db, Transaction

from datetime import datetime


analytics_bp = Blueprint('analytics', __name__)


@analytics_bp.route('/api/category-breakdown')

@login_required

def category_breakdown():

    current_month = datetime.now().strftime('%Y-%m')

    results = db.session.query(

        Category.name,

        db.func.sum(Transaction.amount).label('total')

    ).join(Transaction, Transaction.category_id == Category.id)

    .filter(

        Transaction.user_id == current_user.id,

        Transaction.transaction_type == 'expense',

        db.func.strftime('%Y-%m', Transaction.date) == current_month

    ).group_by(Category.name).all()
```

```
data = [{ 'category': r.name, 'total': float(r.total)} for r in results]
```

```
return jsonify(data)
```

Complete Source Code

Complete source code listed below module wise

app.py

```
import os, csv, io, json
from datetime import date, timedelta
from decimal import Decimal
from functools import wraps

from flask import (Flask, render_template, redirect, url_for, request,
                  flash, jsonify, session, Response, abort)
from flask_sqlalchemy import SQLAlchemy
from flask_login import (LoginManager, UserMixin, login_user, logout_user,
                        login_required, current_user)
from flask_wtf import FlaskForm
from flask_wtf.csrf import CSRFProtect
from wtforms import (StringField, PasswordField, SelectField, DecimalField,
                    TextAreaField, DateField, SubmitField)
from wtforms.validators import DataRequired, Email, EqualTo, Length,
NumberRange, Optional
from werkzeug.security import generate_password_hash, check_password_hash
from sqlalchemy import func

#


---


# APP CONFIG
#


---



app = Flask(__name__)
app.config['SECRET_KEY'] = 'flask-expense-tracker-secret-key-change-in-
production'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///expense.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['WTF_CSRF_ENABLED'] = True

db = SQLAlchemy(app)
csrf = CSRFProtect(app)
login_manager = LoginManager(app)
```

```
login_manager.login_view = 'login'
login_manager.login_message = 'Please log in to access this page.'
login_manager.login_message_category = 'warning'
```

```
#
```

```
—
# MODELS
```

```
#
```

```
—

class User(UserMixin, db.Model):
    __tablename__ = 'user'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    email = db.Column(db.String(254), unique=True, nullable=False)
    password_hash = db.Column(db.String(256), nullable=False)
    preferred_currency = db.Column(db.String(10), default='INR')
    date_joined = db.Column(db.DateTime, default=db.func.now())
    is_admin = db.Column(db.Boolean, default=False, nullable=False)

    expenses = db.relationship('Expense', backref='user', lazy=True, cascade='all,
delete-orphan')
    budgets = db.relationship('Budget', backref='user', lazy=True, cascade='all,
delete-orphan')

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

class Category(db.Model):
    __tablename__ = 'category'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), unique=True, nullable=False)
    icon = db.Column(db.String(50), default='fa-tag')
    color_hex = db.Column(db.String(7), default='#e74c3c')

    expenses = db.relationship('Expense', backref='category', lazy=True)
    budgets = db.relationship('Budget', backref='category', lazy=True)
```

```

def __repr__(self):
    return self.name

class Expense(db.Model):
    __tablename__ = 'expense'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    category_id = db.Column(db.Integer, db.ForeignKey('category.id'),
    nullable=True)
    amount = db.Column(db.Numeric(10, 2), nullable=False)
    date = db.Column(db.Date, nullable=False, default=date.today)
    description = db.Column(db.Text, default="")
    created_at = db.Column(db.DateTime, default=db.func.now())

    def __repr__(self):
        return f'Expense ₹{self.amount} on {self.date}'

class Budget(db.Model):
    __tablename__ = 'budget'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    category_id = db.Column(db.Integer, db.ForeignKey('category.id'),
    nullable=False)
    monthly_limit = db.Column(db.Numeric(10, 2), nullable=False)
    month = db.Column(db.Date, nullable=False) # Always first day of month

    def get_spent(self):
        result = db.session.query(func.sum(Expense.amount)).filter(
            Expense.user_id == self.user_id,
            Expense.category_id == self.category_id,
            db.extract('year', Expense.date) == self.month.year,
            db.extract('month', Expense.date) == self.month.month
        ).scalar()
        return result or Decimal('0')

    def get_percentage(self):
        if self.monthly_limit > 0:
            return min(int((self.get_spent() / self.monthly_limit) * 100), 100)
        return 0

    def __repr__(self):
        return f'Budget: {self.category.name} {self.month.strftime("%B %Y")}'

```

```
@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))
```

```
#
```

```
—
# FORMS
```

```
#
```

```
—

class RegisterForm(FlaskForm):
    username = StringField('Username', validators=[DataRequired(), Length(3,
150)])
    email = StringField('Email', validators=[DataRequired(), Email()])
    preferred_currency = SelectField('Preferred Currency', choices=[
        ('INR', '₹ INR - Indian Rupee'),
        ('USD', '$ USD - US Dollar'),
        ('EUR', '€ EUR - Euro'),
        ('GBP', '£ GBP - British Pound'),
    ])
    password = PasswordField('Password', validators=[DataRequired(),
Length(min=6)])
    confirm_password = PasswordField('Confirm Password',
validators=[DataRequired(), EqualTo('password')])
    submit = SubmitField('Create Account')
```

```
class LoginForm(FlaskForm):
    username = StringField('Username', validators=[DataRequired()])
    password = PasswordField('Password', validators=[DataRequired()])
    submit = SubmitField('Login')
```

```
class ExpenseForm(FlaskForm):
    amount = DecimalField('Amount (₹)', validators=[DataRequired(),
NumberRange(min=0.01)], places=2)
    category_id = SelectField('Category', coerce=int, validators=[Optional()])
    date = DateField('Date', validators=[DataRequired()], default=date.today)
    description = TextAreaField('Description (optional)', validators=[Optional()])
    submit = SubmitField('Save Expense')
```

```
def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
```

```
self.category_id.choices = [(0, '— Select Category —')] + [
    (c.id, c.name) for c in Category.query.order_by(Category.name).all()
]
```

```
class BudgetForm(FlaskForm):
    category_id = SelectField('Category', coerce=int, validators=[DataRequired()])
    monthly_limit = DecimalField('Monthly Limit (₹)', validators=[DataRequired(),
NumberRange(min=1)], places=2)
    month = DateField('Month (pick any day in the month)',
validators=[DataRequired()], default=date.today)
    submit = SubmitField('Save Budget')
```

```
def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    self.category_id.choices = [
        (c.id, c.name) for c in Category.query.order_by(Category.name).all()
    ]
```

```
class CategoryForm(FlaskForm):
    name = StringField('Category Name', validators=[DataRequired(), Length(2,
100)])
    icon = StringField('Font Awesome Icon Class', validators=[DataRequired()],
default='fa-tag')
    color_hex = StringField('Color (hex)', validators=[DataRequired(), Length(7,
7)], default='#e74c3c')
    submit = SubmitField('Save Category')
```

```
#
```

```
—
# HELPERS
```

```
#
```

```
—
DEFAULT_CATEGORIES = [
    ('Food & Dining', 'fa-utensils', '#e74c3c'),
    ('Rent & Housing', 'fa-home', '#3498db'),
    ('Transportation', 'fa-car', '#f39c12'),
    ('Entertainment', 'fa-film', '#9b59b6'),
    ('Health & Medical', 'fa-heartbeat', '#27ae60'),
    ('Shopping', 'fa-shopping-bag', '#e67e22'),
    ('Education', 'fa-book', '#1abc9c'),
```

```

('Utilities', 'fa-bolt', '#2980b9'),
('Subscriptions', 'fa-credit-card', '#8e44ad'),
('Other', 'fa-tag', '#95a5a6'),
]

def admin_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not current_user.is_authenticated or not current_user.is_admin:
            abort(403)
        return f(*args, **kwargs)
    return decorated

def seed_categories():
    for name, icon, color in DEFAULT_CATEGORIES:
        if not Category.query.filter_by(name=name).first():
            db.session.add(Category(name=name, icon=icon, color_hex=color))
    db.session.commit()

def check_budget_alert(user, expense):
    """Alert if budget is near/exceeded after adding an expense."""
    budgets = Budget.query.filter_by(
        user_id=user.id,
        category_id=expense.category_id
    ).all()
    for b in budgets:
        if b.month.year == expense.date.year and b.month.month ==
expense.date.month:
            pct = b.get_percentage()
            if pct >= 100:
                flash(f'⚠ Budget EXCEEDED for {b.category.name}! '
                    f'Spent ₹{b.get_spent()} / Limit ₹{b.monthly_limit}', 'danger')
            elif pct >= 80:
                flash(f'🔔 {pct}% of your {b.category.name} budget used — '
                    f'₹{b.get_spent()} / ₹{b.monthly_limit}', 'warning')

#


---


# ROUTES — AUTH
#


---



```



```

@app.route('/')
def index():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))
    return redirect(url_for('login'))

@app.route('/register', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))
    form = RegisterForm()
    if form.validate_on_submit():
        if User.query.filter_by(username=form.username.data).first():
            flash('Username already taken. Choose another.', 'danger')
            return render_template('register.html', form=form)
        if User.query.filter_by(email=form.email.data).first():
            flash('Email already registered.', 'danger')
            return render_template('register.html', form=form)
        user = User(
            username=form.username.data,
            email=form.email.data,
            preferred_currency=form.preferred_currency.data
        )
        user.set_password(form.password.data)
        if User.query.count() == 0:
            user.is_admin = True
        db.session.add(user)
        db.session.commit()
        seed_categories()
        login_user(user)
        flash(f'Welcome, {user.username}! Start tracking your expenses.', 'success')
        return redirect(url_for('dashboard'))
    return render_template('register.html', form=form)

@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))
    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.data).first()
        if user and user.check_password(form.password.data):

```

```
        login_user(user)
        next_page = request.args.get('next')
        return redirect(next_page or url_for('dashboard'))
    flash('Invalid username or password.', 'danger')
    return render_template('login.html', form=form)
```

```
@app.route('/logout')
@login_required
def logout():
    logout_user()
    flash('You have been logged out.', 'info')
    return redirect(url_for('login'))
```

```
#
```

```
# ROUTES — DASHBOARD
```

```
#
```

```
@app.route('/dashboard')
@login_required
def dashboard():
    today = date.today()

    total_month = db.session.query(func.sum(Expense.amount)).filter(
        Expense.user_id == current_user.id,
        db.extract('year', Expense.date) == today.year,
        db.extract('month', Expense.date) == today.month
    ).scalar() or Decimal('0')

    total_today = db.session.query(func.sum(Expense.amount)).filter(
        Expense.user_id == current_user.id,
        Expense.date == today
    ).scalar() or Decimal('0')

    total_week_start = today - timedelta(days=today.weekday())
    total_week = db.session.query(func.sum(Expense.amount)).filter(
        Expense.user_id == current_user.id,
        Expense.date >= total_week_start,
        Expense.date <= today
    ).scalar() or Decimal('0')
```

```
recent = Expense.query.filter_by(user_id=current_user.id)\
    .order_by(Expense.date.desc(), Expense.created_at.desc()).limit(5).all()
```

```
budgets = Budget.query.filter(
    Budget.user_id == current_user.id,
    db.extract('year', Budget.month) == today.year,
    db.extract('month', Budget.month) == today.month
).all()
```

```
return render_template('dashboard.html',
    total_month=total_month,
    total_today=total_today,
    total_week=total_week,
    recent_expenses=recent,
    budgets=budgets,
    current_month=today.strftime('%B %Y')
)
```

```
#
```

```
# ROUTES — CHART JSON ENDPOINTS
```

```
#
```

```
@app.route('/api/chart/pie')
@login_required
def chart_pie():
    today = date.today()
    rows = db.session.query(Category.name, func.sum(Expense.amount))\
        .join(Expense, Expense.category_id == Category.id)\
        .filter(
            Expense.user_id == current_user.id,
            db.extract('year', Expense.date) == today.year,
            db.extract('month', Expense.date) == today.month
        ).group_by(Category.name).all()
```

```
data = [['Category', 'Amount']] + [[r[0], float(r[1])] for r in rows]
return jsonify({'data': data})
```

```
@app.route('/api/chart/line')
```

```

@login_required
def chart_line():
    today = date.today()
    chart_data = [['Month', 'Expenses']]
    for i in range(5, -1, -1):
        m = (today.replace(day=1) - timedelta(days=i * 28)).replace(day=1)
        total = db.session.query(func.sum(Expense.amount)).filter(
            Expense.user_id == current_user.id,
            db.extract('year', Expense.date) == m.year,
            db.extract('month', Expense.date) == m.month
        ).scalar() or 0
        chart_data.append([m.strftime('%b %Y'), float(total)])
    return jsonify({'data': chart_data})

```

```

@app.route('/api/chart/bar')
@login_required
def chart_bar():
    today = date.today()
    week_start = today - timedelta(days=today.weekday())
    chart_data = [['Day', 'Spent']]
    for i in range(7):
        day = week_start + timedelta(days=i)
        total = db.session.query(func.sum(Expense.amount)).filter(
            Expense.user_id == current_user.id,
            Expense.date == day
        ).scalar() or 0
        chart_data.append([day.strftime('%A'), float(total)])
    return jsonify({'data': chart_data})

```

#

ROUTES — EXPENSES

#

```

@app.route('/expenses')
@login_required
def expenses():
    query = Expense.query.filter_by(user_id=current_user.id)
    category_id = request.args.get('category')
    date_from = request.args.get('date_from')

```

```

date_to = request.args.get('date_to')

if category_id:
    query = query.filter_by(category_id=int(category_id))
if date_from:
    query = query.filter(Expense.date >= date_from)
if date_to:
    query = query.filter(Expense.date <= date_to)

exps = query.order_by(Expense.date.desc(), Expense.created_at.desc()).all()
categories = Category.query.order_by(Category.name).all()
return render_template('expenses.html', expenses=exps, categories=categories)

@app.route('/expenses/add', methods=['GET', 'POST'])
@login_required
def add_expense():
    form = ExpenseForm()
    if form.validate_on_submit():
        exp = Expense(
            user_id=current_user.id,
            amount=form.amount.data,
            category_id=form.category_id.data if form.category_id.data != 0 else None,
            date=form.date.data,
            description=form.description.data or "
        )
        db.session.add(exp)
        db.session.commit()
        check_budget_alert(current_user, exp)
        flash('Expense added successfully!', 'success')
        return redirect(url_for('expenses'))
    return render_template('expense_form.html', form=form, action='Add')

@app.route('/expenses/edit/<int:pk>', methods=['GET', 'POST'])
@login_required
def edit_expense(pk):
    exp = Expense.query.filter_by(id=pk, user_id=current_user.id).first_or_404()
    form = ExpenseForm(obj=exp)
    if form.validate_on_submit():
        exp.amount = form.amount.data
        exp.category_id = form.category_id.data if form.category_id.data != 0 else
None
        exp.date = form.date.data
        exp.description = form.description.data or "

```

```

        db.session.commit()
        flash('Expense updated!', 'success')
        return redirect(url_for('expenses'))
    if exp.category_id:
        form.category_id.data = exp.category_id
    return render_template('expense_form.html', form=form, action='Edit')

@app.route('/expenses/delete/<int:pk>', methods=['GET', 'POST'])
@login_required
def delete_expense(pk):
    exp = Expense.query.filter_by(id=pk, user_id=current_user.id).first_or_404()
    if request.method == 'POST':
        db.session.delete(exp)
        db.session.commit()
        flash('Expense deleted.', 'info')
        return redirect(url_for('expenses'))
    return render_template('confirm_delete.html', obj=exp, type='Expense')

```

#

—

ROUTES — BUDGETS

#

—

```

@app.route('/budgets')
@login_required
def budgets():
    all_budgets = Budget.query.filter_by(user_id=current_user.id)\
        .order_by(Budget.month.desc()).all()
    return render_template('budgets.html', budgets=all_budgets)

```

```

@app.route('/budgets/add', methods=['GET', 'POST'])
@login_required
def add_budget():
    form = BudgetForm()
    if form.validate_on_submit():
        month_first = form.month.data.replace(day=1)
        budget = Budget(
            user_id=current_user.id,
            category_id=form.category_id.data,
            monthly_limit=form.monthly_limit.data,

```

```

        month=month_first
    )
    db.session.add(budget)
    db.session.commit()
    flash('Budget set successfully!', 'success')
    return redirect(url_for('budgets'))
return render_template("budget_form.html", form=form, action='Set')

@app.route('/budgets/delete/<int:pk>', methods=['POST'])
@login_required
def delete_budget(pk):
    budget = Budget.query.filter_by(id=pk, user_id=current_user.id).first_or_404()
    db.session.delete(budget)
    db.session.commit()
    flash('Budget removed.', 'info')
    return redirect(url_for('budgets'))

#


---


# ROUTES — EXPORT
#


---



@app.route('/export/csv')
@login_required
def export_csv():
    exps = Expense.query.filter_by(user_id=current_user.id)\
        .order_by(Expense.date.desc()).all()
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(['Date', 'Category', 'Amount (INR)', 'Description'])
    for e in exps:
        writer.writerow([
            e.date,
            e.category.name if e.category else 'N/A',
            float(e.amount), e.description
        ])
    output.seek(0)
    return Response(
        output.getvalue(),
        mimetype='text/csv',

```

```

        headers={'Content-Disposition': f'attachment;
filename=expenses_{date.today()}.csv'}
    )

@app.route('/export/pdf')
@login_required
def export_pdf():
    try:
        from reportlab.lib.pagesizes import A4
        from reportlab.lib import colors
        from reportlab.platypus import SimpleDocTemplate, Table, TableStyle,
Paragraph, Spacer
        from reportlab.lib.styles import getSampleStyleSheet
        from reportlab.lib.units import inch
    except ImportError:
        flash('ReportLab not installed. Run: pip install reportlab', 'danger')
        return redirect(url_for('dashboard'))

    buffer = io.BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=A4)
    styles = getSampleStyleSheet()
    elements = []

    today = date.today()
    elements.append(Paragraph('Expense Report', styles['Title']))
    elements.append(Paragraph(
        f'Generated: {today.strftime("%d %B %Y")} | User:
{current_user.username}',
        styles['Normal']
    ))
    elements.append(Spacer(1, 0.3 * inch))

    total = db.session.query(func.sum(Expense.amount)).filter(
        Expense.user_id == current_user.id).scalar() or 0

    elements.append(Paragraph(f'Total Expenses (All Time): ₹{total}',
styles['Normal']))
    elements.append(Spacer(1, 0.2 * inch))

    exps = Expense.query.filter_by(user_id=current_user.id)\
        .order_by(Expense.date.desc()).limit(50).all()

    data = [['Date', 'Category', 'Amount (₹)', 'Description']]

```



```

for e in exps:
    data.append([
        str(e.date),
        e.category.name if e.category else 'N/A',
        str(e.amount),
        (e.description or "")[:40]
    ])

table = Table(data, colWidths=[1*inch, 1.4*inch, 1*inch, 3*inch])
table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#e74c3c')),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.white),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('GRID', (0, 0), (-1, -1), 0.5, colors.grey),
    ('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.white,
colors.HexColor('#f8f9fa')]),
    ('FONTSIZE', (0, 0), (-1, -1), 9),
]))
elements.append(table)
doc.build(elements)
buffer.seek(0)
return Response(
    buffer.getvalue(),
    mimetype='application/pdf',
    headers={'Content-Disposition': f'attachment;
filename=expenses_{today}.pdf'}
)

```

#

ROUTES — ADMIN PANEL

#

```

@app.route('/admin')
@login_required
@admin_required
def admin_dashboard():
    total_users = User.query.count()
    total_expenses = Expense.query.count()
    total_spent = db.session.query(func.sum(Expense.amount)).scalar() or 0

```

```

total_budgets = Budget.query.count()
recent_users = User.query.order_by(User.date_joined.desc()).limit(5).all()
recent_exps = Expense.query.order_by(Expense.created_at.desc()).limit(10).all()
return render_template('admin_dashboard.html',
    total_users=total_users,
    total_expenses=total_expenses,
    total_spent=total_spent,
    total_budgets=total_budgets,
    recent_users=recent_users,
    recent_exps=recent_exps,
)

@app.route('/admin/users')
@login_required
@admin_required
def admin_users():
    users = User.query.order_by(User.date_joined.desc()).all()
    return render_template('admin_users.html', users=users)

@app.route('/admin/users/<int:pk>/toggle-admin', methods=['POST'])
@login_required
@admin_required
def admin_toggle_admin(pk):
    user = User.query.get_or_404(pk)
    if user.id == current_user.id:
        flash("You cannot change your own admin status.", 'warning')
    else:
        user.is_admin = not user.is_admin
        db.session.commit()
        flash(f'{user.username} Admin granted to' if user.is_admin else 'Admin revoked from')
    return redirect(url_for('admin_users'))

@app.route('/admin/users/<int:pk>/delete', methods=['POST'])
@login_required
@admin_required
def admin_delete_user(pk):
    user = User.query.get_or_404(pk)
    if user.id == current_user.id:
        flash("You cannot delete yourself.", 'danger')
    else:
        db.session.delete(user)
        db.session.commit()

```

```

        flash(f"User '{user.username}' deleted.", 'info')
    return redirect(url_for('admin_users'))

@app.route('/admin/expenses')
@login_required
@admin_required
def admin_expenses():
    exps = Expense.query.order_by(Expense.date.desc(),
Expense.created_at.desc()).all()
    return render_template('admin_expenses.html', expenses=exps)

@app.route('/admin/expenses/<int:pk>/delete', methods=['POST'])
@login_required
@admin_required
def admin_delete_expense(pk):
    exp = Expense.query.get_or_404(pk)
    db.session.delete(exp)
    db.session.commit()
    flash('Expense deleted.', 'info')
    return redirect(url_for('admin_expenses'))

@app.route('/admin/categories')
@login_required
@admin_required
def admin_categories():
    cats = Category.query.order_by(Category.name).all()
    form = CategoryForm()
    return render_template('admin_categories.html', categories=cats, form=form)

@app.route('/admin/categories/add', methods=['POST'])
@login_required
@admin_required
def admin_add_category():
    form = CategoryForm()
    if form.validate_on_submit():
        if Category.query.filter_by(name=form.name.data).first():
            flash('Category already exists.', 'warning')
        else:
            cat = Category(name=form.name.data, icon=form.icon.data,
color_hex=form.color_hex.data)
            db.session.add(cat)
            db.session.commit()
            flash(f"Category '{cat.name}' added.", 'success')

```

```

else:
    for field, errs in form.errors.items():
        for e in errs:
            flash(f"{field}: {e}", 'danger')
    return redirect(url_for('admin_categories'))

@app.route('/admin/categories/<int:pk>/delete', methods=['POST'])
@login_required
@admin_required
def admin_delete_category(pk):
    cat = Category.query.get_or_404(pk)
    if cat.expenses:
        flash(f"Cannot delete '{cat.name}' — it has expenses linked.", 'danger')
    else:
        db.session.delete(cat)
        db.session.commit()
        flash(f"Category '{cat.name}' deleted.", 'info')
    return redirect(url_for('admin_categories'))

```

#

CONTEXT PROCESSOR

#

```

@app.context_processor
def inject_now():
    return {'now': date.today()}

```

#

INIT DB & RUN

#

```

with app.app_context():
    db.create_all()
    seed_categories()

```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

Requirements.txt

Flask>=3.0.0

Flask-SQLAlchemy>=3.1.0

Flask-Login>=0.6.3

Flask-WTF>=1.2.1

WTForms>=3.1.1

email-validator>=2.1.0

Werkzeug>=3.0.0

reportlab>=4.0.1

expenses.db

-- USER TABLE

CREATE TABLE user (

id INTEGER PRIMARY KEY,

username VARCHAR(150) NOT NULL UNIQUE,

email VARCHAR(254) NOT NULL UNIQUE,

password_hash VARCHAR(256) NOT NULL,

preferred_currency VARCHAR(10),

date_joined DATETIME,

is_admin BOOLEAN NOT NULL

);

-- CATEGORY TABLE

```
CREATE TABLE category (  
  
    id INTEGER PRIMARY KEY,  
  
    name VARCHAR(100) NOT NULL UNIQUE,  
  
    icon VARCHAR(50),  
  
    color_hex VARCHAR(7)  
  
);
```

-- EXPENSE TABLE

```
CREATE TABLE expense (  
  
    id INTEGER PRIMARY KEY,  
  
    user_id INTEGER NOT NULL,  
  
    category_id INTEGER,  
  
    amount NUMERIC(10,2) NOT NULL,  
  
    date DATE NOT NULL,  
  
    description TEXT,
```

```
created_at DATETIME,  
  
FOREIGN KEY (user_id) REFERENCES user(id),  
  
FOREIGN KEY (category_id) REFERENCES category(id)  
  
);
```

-- BUDGET TABLE

```
CREATE TABLE budget (  
  
    id INTEGER PRIMARY KEY,  
  
    user_id INTEGER NOT NULL,  
  
    category_id INTEGER NOT NULL,  
  
    monthly_limit NUMERIC(10,2) NOT NULL,  
  
    month DATE NOT NULL,  
  
    FOREIGN KEY (user_id) REFERENCES user(id),  
  
    FOREIGN KEY (category_id) REFERENCES category(id)  
  
);
```

Admin_categories.html

```
{% extends 'base.html' %}
```

{% block title %}Admin — Categories{% endblock %}

{% block content %}

<div class="d-flex align-items-center gap-3 mb-4">

<div>

<h4 class="fw-bold mb-0"><i class="fas fa-shield-alt me-2" style="color:#f39c12;"></i>Admin Panel</h4>

<small class="text-muted">Manage expense categories</small>

</div>

</div>

<ul class="nav nav-pills mb-4" style="gap:8px;">

<li class="nav-item"><i class="fas fa-tachometer-alt me-1"></i>Overview

<li class="nav-item"><i class="fas fa-users me-1"></i>Users

<li class="nav-item"><i class="fas fa-money-bill-wave me-1"></i>All Expenses


```
<li class="nav-item"><a class="nav-link active" href="{ {
url_for('admin_categories') }}"><i class="fas fa-tags me-
1"></i>Categories</a></li>

</ul>
```

```
<div class="row g-3">
```

```
<!-- Add Category Form -->
```

```
<div class="col-md-4">
```

```
<div class="card p-4">
```

```
<h6 class="fw-bold mb-3"><i class="fas fa-plus-circle me-2 text-
danger"></i>Add New Category</h6>
```

```
<form method="POST" action="{ { url_for('admin_add_category') }}">
```

```
{ { form.hidden_tag() } }
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold small">{ { form.name.label.text
}}</label>
```

```
{ { form.name(class="form-control form-control-sm", placeholder="e.g.
Groceries") } }
```

```
    {% for e in form.name.errors %}<div class="text-danger small">{{ e
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
    <label class="form-label fw-semibold small">{{ form.icon.label.text
}}</label>
```

```
    {{ form.icon(class="form-control form-control-sm", placeholder="fa-tag") }}
```

```
    <small class="text-muted">From <a href="https://fontawesome.com/icons"
target="_blank">Font Awesome</a> e.g. fa-car</small>
```

```
    {% for e in form.icon.errors %}<div class="text-danger small">{{ e
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
    <label class="form-label fw-semibold small">{{ form.color_hex.label.text
}}</label>
```

```
<div class="input-group input-group-sm">
```

```
    <input type="color" class="form-control form-control-sm" style="max-
width:50px;padding:2px;"
```

```
        oninput="document.getElementById('colorText').value=this.value"
        value="#e74c3c">
```

```
        {{ form.color_hex(class="form-control form-control-sm", id="colorText")
    }}
```

```
</div>
```

```
        {% for e in form.color_hex.errors %}<div class="text-danger small">{{ e
    }}</div>{% endfor %}
```

```
</div>
```

```
        {{ form.submit(class="btn btn-danger btn-sm w-100") }}
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<!-- Category List -->
```

```
<div class="col-md-8">
```

```
<div class="card">
```

```
<div class="card-body p-0">
```

```
<table class="table table-hover mb-0">
```

```
<thead class="table-dark">
```

```
<tr><th>Preview</th><th>Name</th><th>Icon
```

```
Class</th><th>Expenses</th><th>Action</th></tr>
```

```
</thead>
```

```
<tbody>
```

```
{% for cat in categories %}
```

```
<tr>
```

```
<td>
```

```
    <span class="badge rounded-pill" style="background:{{ cat.color_hex }};
font-size:.85rem; padding:6px 12px;">
```

```
        <i class="fas {{ cat.icon }} me-1"></i>{{ cat.name }}
```

```
    </span>
```

```
</td>
```

```
<td class="fw-semibold">{{ cat.name }}</td>
```

```
<td><code class="small">{{ cat.icon }}</code></td>
```

```
<td class="text-center">
```

```
    <span class="badge bg-light text-dark">{{ cat.expenses | length }}</span>
```

```
</td>
```

```
<td>
```

```
{% if cat.expenses | length == 0 %}
```

```
    <form method="POST" action="{{ url_for('admin_delete_category',
pk=cat.id) }}" class="d-inline"
```

```
onsubmit="return confirm('Delete category {{ cat.name }}?')">
```

```
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

```
        <button class="btn btn-sm btn-outline-danger" title="Delete"><i
class="fas fa-trash"></i></button>
```

```
</form>
```

```
{% else %}
```

```
        <span class="text-muted small" title="Has linked expenses"><img alt="lock icon" data-bbox="858 333 881 353"/> In
use</span>
```

```
{% endif %}
```

```
</td>
```

```
</tr>
```

```
{% else %}
```

```
        <tr><td colspan="5" class="text-center text-muted py-4">No categories
found.</td></tr>
```

```
{% endfor %}
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

</div>

</div>

{% endblock %}

Admin_dashboard.html

{% extends 'base.html' %}

{% block title %}Admin — Overview{% endblock %}

{% block content %}

<div class="d-flex align-items-center gap-3 mb-4">

<div>

<h4 class="fw-bold mb-0"><i class="fas fa-shield-alt me-2" style="color:#f39c12;"></i>Admin Panel</h4>

<small class="text-muted">Platform-wide overview and management</small>

</div>

</div>

<ul class="nav nav-pills mb-4" style="gap:8px;">

```
<li class="nav-item"><a class="nav-link active" href="{ { url_for('admin_dashboard') } }"><i class="fas fa-tachometer-alt me-1"></i>Overview</a></li>
```

```
<li class="nav-item"><a class="nav-link" href="{ { url_for('admin_users') } }"><i class="fas fa-users me-1"></i>Users</a></li>
```

```
<li class="nav-item"><a class="nav-link" href="{ { url_for('admin_expenses') } }"><i class="fas fa-money-bill-wave me-1"></i>All Expenses</a></li>
```

```
<li class="nav-item"><a class="nav-link" href="{ { url_for('admin_categories') } }"><i class="fas fa-tags me-1"></i>Categories</a></li>
```

```
</ul>
```

```
<!-- STAT CARDS -->
```

```
<div class="row g-3 mb-4">
```

```
<div class="col-md-3">
```

```
<div class="stat-card h-100" style="background:linear-gradient(135deg,#8e44ad,#9b59b6);">
```

```
<div class="label"><i class="fas fa-users me-1"></i>Total Users</div>
```

```
<div class="value">{ { total_users } }</div>
```

```
</div>
```

</div>

<div class="col-md-3">

<div class="stat-card h-100" style="background:linear-gradient(135deg,#e74c3c,#c0392b);">

<div class="label"><i class="fas fa-money-bill-wave me-1"></i>Total Expenses</div>

<div class="value">{{ total_expenses }}</div>

</div>

</div>

<div class="col-md-3">

<div class="stat-card h-100" style="background:linear-gradient(135deg,#e67e22,#d35400);">

<div class="label"><i class="fas fa-rupee-sign me-1"></i>Platform Total Spent</div>

<div class="value" style="font-size:1.4rem;">₹{{ '%.0f'|format(total_spent|float) }}</div>

</div>

</div>

<div class="col-md-3">


```
<div class="stat-card h-100" style="background:linear-gradient(135deg,#27ae60,#2ecc71);">
```

```
<div class="label"><i class="fas fa-piggy-bank me-1"></i>Active  
Budgets</div>
```

```
<div class="value">{{ total_budgets }}</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="row g-3">
```

```
<!-- Recent Users -->
```

```
<div class="col-md-5">
```

```
<div class="card p-3 h-100">
```

```
<div class="d-flex justify-content-between align-items-center mb-3">
```

```
<h6 class="fw-bold mb-0"><i class="fas fa-user-clock me-2" style="color:#9b59b6;"></i>Recent Signups</h6>
```

```
<a href="{{ url_for('admin_users') }}" class="btn btn-sm btn-outline-secondary">View All</a>
```

```
</div>
```

```

<table class="table table-sm table-hover mb-0">

                                <thead                                class="table-
light"><tr><th>Username</th><th>Joined</th><th>Role</th></tr></thead>

<tbody>

    {% for u in recent_users %}

    <tr>

        <td class="fw-semibold">{{ u.username }}</td>

        <td class="small text-muted">{{ u.date_joined.strftime('%d %b %Y') if
u.date_joined else '—' }}</td>

        <td>

            {% if u.is_admin %}

                <span class="badge" style="background:#f39c12;">Admin</span>

            {% else %}

                <span class="badge bg-secondary">User</span>

            {% endif %}

        </td>

    </tr>

    {% endfor %}

</tbody>

```

</table>

</div>

</div>

<!-- Recent Expenses -->

<div class="col-md-7">

<div class="card p-3 h-100">

<div class="d-flex justify-content-between align-items-center mb-3">

<h6 class="fw-bold mb-0"><i class="fas fa-history me-2 text-info"></i>Latest
Expenses</h6>

<a href="{ { url_for('admin_expenses') } }" class="btn btn-sm btn-outline-
secondary">View All

</div>

<table class="table table-sm table-hover mb-0">

<thead class="table-
light"><tr><th>User</th><th>Date</th><th>Category</th><th>Amount</th></tr>
></thead>

<tbody>

{% for e in recent_exps %}

```

<tr>

    <td class="small">{{ e.user.username }}</td>

    <td class="small text-muted">{{ e.date }}</td>

    <td>

        <span class="badge badge-expense">{{ e.category.name if e.category else
'—' }}</span>

    </td>

    <td class="fw-semibold text-danger">-₹{{ e.amount }}</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

</div>

{% endblock %}

```

Admin_expenses.html

```

{% extends 'base.html' %}

{% block title %}Admin — All Expenses{% endblock %}

```

{% block content %}

<div class="d-flex align-items-center gap-3 mb-4">

<div>

<h4 class="fw-bold mb-0"><i class="fas fa-shield-alt me-2" style="color:#f39c12;"></i>Admin Panel</h4>

<small class="text-muted">All expenses across all users</small>

</div>

</div>

<ul class="nav nav-pills mb-4" style="gap:8px;">

<li class="nav-item"><i class="fas fa-tachometer-alt me-1"></i>Overview

<li class="nav-item"><i class="fas fa-users me-1"></i>Users

<li class="nav-item"><i class="fas fa-money-bill-wave me-1"></i>All Expenses

```
<li class="nav-item"><a class="nav-link" href="{{ url_for('admin_categories')
}}"><i class="fas fa-tags me-1"></i>Categories</a></li>

</ul>
```

```
<div class="card">
```

```
<div class="card-body p-0">
```

```
<table class="table table-hover mb-0">
```

```
<thead class="table-dark">
```

```
<tr>
```

```
<th>ID</th><th>User</th><th>Date</th><th>Category</th>
```

```
<th>Amount</th><th>Description</th><th>Action</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{% for e in expenses %}
```

```
<tr>
```

```
<td class="text-muted small">#{{ e.id }}</td>
```

```
<td class="fw-semibold small">{{ e.user.username }}</td>
```

```
<td class="small">{{ e.date }}</td>
```

```

<td class="small">{{ e.category.name if e.category else '—' }}</td>

<td class="fw-bold text-danger">-₹{{ e.amount }}</td>

<td class="small text-muted">{{ (e.description or '—')[:35] }}</td>

<td>

    <form method="POST" action="{{ url_for('admin_delete_expense', pk=e.id)
}} " class="d-inline"

        onsubmit="return confirm('Delete this expense permanently?')">

            <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

            <button class="btn btn-sm btn-outline-danger" title="Delete"><i class="fas
fa-trash"></i></button>

        </form>

    </td>

</tr>

{% else %}

    <tr><td colspan="7" class="text-center text-muted py-4">No expenses
found.</td></tr>

{% endfor %}

</tbody>

</table>

```

</div>

</div>

{% endblock %}

Admin_user.html

{% extends 'base.html' %}

{% block title %}Admin — Users{% endblock %}

{% block content %}

<div class="d-flex align-items-center gap-3 mb-4">

<div>

<h4 class="fw-bold mb-0"><i class="fas fa-shield-alt me-2"

style="color:#f39c12;"></i>Admin Panel</h4>

<small class="text-muted">Manage registered users</small>

</div>

</div>

<ul class="nav nav-pills mb-4" style="gap:8px;">

<li class="nav-item"><a class="nav-link" href="{ { url_for('admin_dashboard')
}}"><i class="fas fa-tachometer-alt me-1"></i>Overview


```
<li class="nav-item"><a class="nav-link active" href="{{ url_for('admin_users')
}}"><i class="fas fa-users me-1"></i>Users</a></li>

<li class="nav-item"><a class="nav-link" href="{{ url_for('admin_expenses')
}}"><i class="fas fa-money-bill-wave me-1"></i>All Expenses</a></li>

<li class="nav-item"><a class="nav-link" href="{{ url_for('admin_categories')
}}"><i class="fas fa-tags me-1"></i>Categories</a></li>

</ul>
```

```
<div class="card">
```

```
<div class="card-body p-0">
```

```
<table class="table table-hover mb-0">
```

```
<thead class="table-dark">
```

```
<tr>
```

```
<th>ID</th><th>Username</th><th>Email</th><th>Currency</th>
```

```
<th>Joined</th><th>Expenses</th><th>Role</th><th>Actions</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{% for u in users %}
```

```

<tr {% if u.id == current_user.id %}class="table-warning"{% endif %}>

    <td class="text-muted small">#{ { u.id } }</td>

    <td class="fw-semibold">

        { { u.username } }

        {% if u.id == current_user.id %}<span class="badge bg-secondary ms-
1">You</span>{% endif %}

    </td>

    <td class="small">{ { u.email } }</td>

        <td><span class="badge" style="background:#e74c3c;">{ {
u.preferred_currency } }</span></td>

        <td class="small text-muted">{ { u.date_joined.strftime('%d %b %Y') if
u.date_joined else '—' } }</td>

    <td class="text-center">{ { u.expenses | length } }</td>

    <td>

        {% if u.is_admin %}

            <span class="badge" style="background:#f39c12;"><i class="fas fa-shield-
alt me-1"></i>Admin</span>

        {% else %}

            <span class="badge bg-secondary">User</span>

```

```

        {% endif %}

    </td>

    <td>

        {% if u.id != current_user.id %}

            <form method="POST" action="{{ url_for('admin_toggle_admin', pk=u.id)
}} " class="d-inline">

                <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

                <button class="btn btn-sm {% if u.is_admin %}btn-outline-warning{% else
}%}btn-outline-success{% endif %} me-1"

                    title="{{ 'Revoke Admin' if u.is_admin else 'Grant Admin' }}">

                        <i class="fas {% if u.is_admin %}fa-user-minus{% else %}fa-user-
shield{% endif %}"></i>

                    </button>

                </form>

                <form method="POST" action="{{ url_for('admin_delete_user', pk=u.id) }}"
class="d-inline"

                    onsubmit="return confirm('Delete user {{ u.username }} and ALL their
data?')">

                    <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

```

```

        <button class="btn btn-sm btn-outline-danger" title="Delete User">

            <i class="fas fa-trash"></i>

        </button>

    </form>

    {% else %}

        <span class="text-muted small">—</span>

    {% endif %}

</td>

</tr>

{% else %}

    <tr><td colspan="8" class="text-center text-muted py-4">No users
found.</td></tr>

    {% endfor %}

</tbody>

</table>

</div>

</div>

{% endblock %}

```

Base.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>{% block title %}Expense Tracker{% endblock %}</title>
```

```
<link
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
```

```
rel="stylesheet">
```

```
<link
```

```
href="https://cdnjs.cloudflare.com/ajax/libs/font-
```

```
awesome/6.4.0/css/all.min.css" rel="stylesheet">
```

```
<style>
```

```
:root {
```

```
--primary: #2c3e50;
```

```
--accent: #e74c3c;
```

```
}
```

```
body { background: #f0f2f5; font-family: 'Segoe UI', sans-serif; }
```

```
/* — SIDEBAR — */
```

```
.sidebar {  
  
  width: 240px; min-height: 100vh; background: var(--primary);  
  
  position: fixed; top: 0; left: 0; z-index: 100;  
  
  display: flex; flex-direction: column;  
  
}  
  
.sidebar-brand { padding: 20px; color: #fff; font-size: 1.1rem; font-weight: 700;  
  
  border-bottom: 1px solid rgba(255,255,255,.1); }  
  
.sidebar-brand i { color: var(--accent); margin-right: 8px; }  
  
.sidebar .nav-link { color: rgba(255,255,255,.75); padding: 12px 20px; transition:  
all .2s; }  
  
.sidebar .nav-link:hover,  
  
.sidebar .nav-link.active { color: #fff; background: rgba(255,255,255,.1); padding-  
left: 28px; }  
  
.sidebar .nav-link i { width: 20px; margin-right: 8px; }  
  
.main-content { margin-left: 240px; padding: 24px; min-height: 100vh; }  
  
/* — TOPBAR — */  
  
.topbar { background: #fff; border-radius: 12px; padding: 14px 24px;
```

```
margin-bottom: 24px; box-shadow: 0 2px 8px rgba(0,0,0,.06);
```

```
display: flex; justify-content: space-between; align-items: center; }
```

```
/* — CARDS — */
```

```
.card { border: none; border-radius: 12px; box-shadow: 0 2px 10px  
rgba(0,0,0,.07); }
```

```
.stat-card { border-radius: 12px; padding: 20px; color: #fff; }
```

```
.stat-card .value { font-size: 1.8rem; font-weight: 700; }
```

```
.stat-card .label { font-size: .85rem; opacity: .85; }
```

```
.badge-expense { background: #fde8e8; color: #c0392b; }
```

```
@media (max-width: 768px) {
```

```
.sidebar { width: 100%; min-height: auto; position: relative; }
```

```
.main-content { margin-left: 0; }
```

```
}
```

```
</style>
```

```
{% block extra_head %}{% endblock %}
```

```
</head>
```

<body>

{% if current_user.is_authenticated %}

<div class="sidebar">

<div class="sidebar-brand"><i class="fas fa-receipt"></i>ExpenseTracker</div>

<nav class="nav flex-column mt-2">

<a class="nav-link {% if request.endpoint == 'dashboard' %}active{% endif %}"
href="{{ url_for('dashboard') }}"><i class="fas fa-tachometer-
alt"></i>Dashboard

<a class="nav-link {% if 'expense' in (request.endpoint or '') %}active{% endif
%}"

href="{{ url_for('expenses') }}"><i class="fas fa-money-bill-
wave"></i>Expenses

<a class="nav-link {% if 'budget' in (request.endpoint or '') %}active{% endif %}"

href="{{ url_for('budgets') }}"><i class="fas fa-piggy-bank"></i>Budgets

<hr style="border-color:rgba(255,255,255,.1);margin:8px 16px;">

<i class="fas fa-file-
csv"></i>Export CSV


```
<a class="nav-link" href="{{ url_for('export_pdf') }}"><i class="fas fa-file-pdf"></i>Export PDF</a>
```

```
{% if current_user.is_admin %}
```

```
<hr style="border-color:rgba(255,255,255,.1);margin:8px 16px;">
```

```
<a class="nav-link {% if 'admin' in (request.endpoint or '') %}active{% endif %}"
```

```
href="{{ url_for('admin_dashboard') }}" style="color:#f39c12;">
```

```
<i class="fas fa-shield-alt"></i>Admin Panel</a>
```

```
{% endif %}
```

```
<hr style="border-color:rgba(255,255,255,.1);margin:8px 16px;">
```

```
<a class="nav-link text-danger" href="{{ url_for('logout') }}"><i class="fas fa-sign-out-alt"></i>Logout</a>
```

```
</nav>
```

```
</div>
```

```
{% endif %}
```

```
<div class="{% if current_user.is_authenticated %}main-content{% else %}container-fluid{% endif %}">
```

```
{% if current_user.is_authenticated %}
```

```
<div class="topbar">
```

```
<div><i class="fas fa-calendar-alt text-muted me-2"></i>
```

```
<strong>{{ now.strftime('%A, %d %B %Y') }}</strong>
```

```
</div>
```

```
<div>
```

```
<span class="text-muted me-2">Hello,</span>
```

```
<strong>{{ current_user.username }}</strong>
```

```
<span class="badge ms-2" style="background:var(--accent)">{{  
current_user.preferred_currency }}</span>
```

```
</div>
```

```
</div>
```

```
{% endif %}
```

```
{% with messages = get_flashed_messages(with_categories=true) %}
```

```
{% for category, message in messages %}
```

```
<div class="alert alert-{{ category }} alert-dismissible fade show" role="alert">
```

```
  {{ message }}
```

```
<button type="button" class="btn-close" data-bs-dismiss="alert"></button>
```

```
</div>
```

```
{% endfor %}

{% endwith %}


{% block content %}{% endblock %}

</div>


<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>

{% block extra_js %}{% endblock %}

</body>

</html>
```

Budget_form.html

```
{% extends 'base.html' %}

{% block title %}{{ action }} Budget{% endblock %}

{% block content %}

<div class="row justify-content-center">

    <div class="col-md-5">

        <div class="card p-4">
```

```
<h5 class="fw-bold mb-4"><i class="fas fa-piggy-bank me-2 text-  
warning"></i>{{ action }} Budget</h5>
```

```
<form method="POST">
```

```
{{ form.hidden_tag() }}
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.category_id.label.text  
}}</label>
```

```
{{ form.category_id(class="form-control") }}
```

```
{% for e in form.category_id.errors %}<div class="text-danger small">{{ e  
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.monthly_limit.label.text  
}}</label>
```

```
{{ form.monthly_limit(class="form-control", placeholder="e.g. 5000") }}
```

```
{% for e in form.monthly_limit.errors %}<div class="text-danger small">{{  
e }}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```

<label class="form-label fw-semibold">{{ form.month.label.text }}</label>

{{ form.month(class="form-control") }}

<small class="text-muted">Pick any date in the target month</small>

{% for e in form.month.errors %}<div class="text-danger small">{{ e
}}</div>{% endfor %}

</div>

<div class="d-flex gap-2 mt-3">

{{ form.submit(class="btn btn-warning flex-grow-1") }}

    <a href="{{ url_for('budgets') }}" class="btn btn-outline-
secondary">Cancel</a>

</div>

</form>

</div>

</div>

</div>

{% endblock %}

```

Budget_form.html

```

{% extends 'base.html' %}

{% block title %}{{ action }} Budget{% endblock %}

```

```
{% block content %}
```

```
<div class="row justify-content-center">
```

```
<div class="col-md-5">
```

```
<div class="card p-4">
```

```
    <h5 class="fw-bold mb-4"><i class="fas fa-piggy-bank me-2 text-  
warning"></i>{{ action }} Budget</h5>
```

```
<form method="POST">
```

```
    {{ form.hidden_tag() }}
```

```
<div class="mb-3">
```

```
    <label class="form-label fw-semibold">{{ form.category_id.label.text  
}}</label>
```

```
    {{ form.category_id(class="form-control") }}
```

```
    {% for e in form.category_id.errors %}<div class="text-danger small">{{ e  
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
    <label class="form-label fw-semibold">{{ form.monthly_limit.label.text  
}}</label>
```

```
    {{ form.monthly_limit(class="form-control", placeholder="e.g. 5000") }}
```

```
        {% for e in form.monthly_limit.errors %}<div class="text-danger small">{{
e }}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.month.label.text }}</label>
```

```
{{ form.month(class="form-control") }}
```

```
<small class="text-muted">Pick any date in the target month</small>
```

```
        {% for e in form.month.errors %}<div class="text-danger small">{{ e
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="d-flex gap-2 mt-3">
```

```
{{ form.submit(class="btn btn-warning flex-grow-1") }}
```

```
        <a href="{{ url_for('budgets') }}" class="btn btn-outline-
secondary">Cancel</a>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

{% endblock %}

Confirm_delete.html

{% extends 'base.html' %}

{% block title %}Confirm Delete{% endblock %}

{% block content %}

<div class="row justify-content-center">

<div class="col-md-5">

<div class="card p-4 text-center">

<i class="fas fa-exclamation-triangle fa-3x text-danger mb-3"></i>

<h5 class="fw-bold">Delete {{ type }}?</h5>

<p class="text-muted">Are you sure you want to delete {{ obj
}}?
This cannot be undone.</p>

<form method="POST" class="d-flex gap-2 justify-content-center mt-3">

<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

<button type="submit" class="btn btn-danger px-4">Yes, Delete</button>

<a href="javascript:history.back()" class="btn btn-outline-secondary px-
4">Cancel

</form>

</div>

</div>

</div>

{% endblock %}

Dashboard.html

{% extends 'base.html' %}

{% block title %}Dashboard – Expense Tracker{% endblock %}

{% block extra_head %}

<script type="text/javascript"

src="https://www.gstatic.com/charts/loader.js"></script>

{% endblock %}

{% block content %}

<h4 class="fw-bold mb-4"><i class="fas fa-tachometer-alt me-2 text-danger"></i>Dashboard — {{ current_month }}</h4>

<!-- STAT CARDS -->

<div class="row g-3 mb-4">

<div class="col-md-4">

```

                <div                class="stat-card"                style="background:linear-
gradient(135deg,#e74c3c,#c0392b);">

```

```

    <div class="label"><i class="fas fa-calendar-alt me-1"></i>This Month</div>

```

```

    <div class="value">₹ {{ "%.2f"|format(total_month|float) }}</div>

```

```

</div>

```

```

</div>

```

```

<div class="col-md-4">

```

```

                <div                class="stat-card"                style="background:linear-
gradient(135deg,#e67e22,#d35400);">

```

```

    <div class="label"><i class="fas fa-sun me-1"></i>Today</div>

```

```

    <div class="value">₹ {{ "%.2f"|format(total_today|float) }}</div>

```

```

</div>

```

```

</div>

```

```

<div class="col-md-4">

```

```

                <div                class="stat-card"                style="background:linear-
gradient(135deg,#8e44ad,#9b59b6);">

```

```

    <div class="label"><i class="fas fa-calendar-week me-1"></i>This Week</div>

```

```

    <div class="value">₹ {{ "%.2f"|format(total_week|float) }}</div>

```

```

</div>

```

</div>

</div>

<!-- CHARTS -->

<div class="row g-3 mb-4">

<div class="col-md-4">

<div class="card p-3 h-100">

<h6 class="fw-bold"><i class="fas fa-chart-pie me-2 text-danger"></i>Spending
by Category</h6>

<div id="pieChart" style="height:260px;"></div>

</div>

</div>

<div class="col-md-8">

<div class="card p-3 h-100">

<h6 class="fw-bold"><i class="fas fa-chart-line me-2 text-
warning"></i>Monthly Expenses (6 Months)</h6>

<div id="lineChart" style="height:260px;"></div>

</div>

</div>

</div>

<div class="row g-3 mb-4">

<div class="col-12">

<div class="card p-3">

<h6 class="fw-bold"><i class="fas fa-chart-bar me-2 text-danger"></i>This
Week's Daily Spending</h6>

<div id="barChart" style="height:220px;"></div>

</div>

</div>

</div>

<!-- BUDGETS & RECENT EXPENSES -->

<div class="row g-3">

<div class="col-md-6">

<div class="card p-3">

<div class="d-flex justify-content-between align-items-center mb-3">

<h6 class="fw-bold mb-0"><i class="fas fa-piggy-bank me-2 text-
warning"></i>Budget Status</h6>

```

        <a href="{{ url_for('add_budget') }}" class="btn btn-sm btn-outline-danger">+
Add Budget</a>

</div>

{% if budgets %}

{% for b in budgets %}

<div class="mb-3">

    <div class="d-flex justify-content-between">

        <span class="small fw-semibold">{{ b.category.name }}</span>

        <span class="small text-muted">₹{{ b.get_spent() }} / ₹{{ b.monthly_limit
    }}</span>

    </div>

    <div class="progress" style="height:8px;border-radius:4px;">

        <div class="progress-bar

            {% if b.get_percentage() >= 100 %}bg-danger

            {% elif b.get_percentage() >= 80 %}bg-warning

            {% else %}bg-success{% endif %}"

            style="width:{{ b.get_percentage() }}%"></div>

    </div>

    <small class="text-muted">{{ b.get_percentage() }}% used</small>

```

</div>

{% endfor %}

{% else %}

<p class="text-muted small">No budgets for this month. Set one</p>

{% endif %}

</div>

</div>

<div class="col-md-6">

<div class="card p-3">

<div class="d-flex justify-content-between align-items-center mb-3">

<h6 class="fw-bold mb-0"><i class="fas fa-history me-2 text-info"></i>Recent Expenses</h6>

+ Add

</div>

{% if recent_expenses %}

<table class="table table-sm table-hover">

```

<thead><tr><th>Date</th><th>Category</th><th>Amount</th></tr></thead>

<tbody>

{% for e in recent_expenses %}

<tr>

<td class="small">{{ e.date }}</td>

<td>

<span class="badge badge-expense">

{{ e.category.name if e.category else '—' }}

</span>

</td>

<td class="fw-semibold text-danger">-₹{{ e.amount }}</td>

</tr>

{% endfor %}

</tbody>

</table>

<a href="{{ url_for('expenses') }}" class="btn btn-sm btn-outline-secondary w-
100">View All</a>

{% else %}

```

```
        <p class="text-muted small">No expenses yet. <a href="{{
url_for('add_expense') }}">Add your first one!</a></p>
```

```
    {% endif %}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

```
{% block extra_js %}
```

```
<script>
```

```
google.charts.load('current', {'packages': ['corechart', 'bar']});
```

```
google.charts.setOnLoadCallback(drawAll);
```

```
function drawAll() { drawPie(); drawLine(); drawBar(); }
```

```
async function drawPie() {
```

```
    const res = await fetch("{{ url_for('chart_pie') }}");
```

```
    const json = await res.json();
```

```
    if (json.data.length <= 1) {
```



```
document.getElementById('pieChart').innerHTML = '<p class="text-muted text-center mt-5">No expense data yet</p>';
```

```
return;
```

```
}
```

```
const data = google.visualization.arrayToDataTable(json.data);
```

new

```
google.visualization.PieChart(document.getElementById('pieChart')).draw(data, {
```

```
  pieHole: 0.4, legend: { position: 'bottom', textStyle: { fontSize: 11 } },
```

```
  chartArea: { width: '90%', height: '75%' }, backgroundColor: 'transparent'
```

```
});
```

```
}
```

```
async function drawLine() {
```

```
  const res = await fetch("{{ url_for('chart_line') }}");
```

```
  const json = await res.json();
```

```
  const data = google.visualization.arrayToDataTable(json.data);
```

new

```
google.visualization.AreaChart(document.getElementById('lineChart')).draw(data,
```

```
{
```

```

    colors: ['#e74c3c'], legend: { position: 'top' },

    chartArea: { width: '88%', height: '75%' }, backgroundColor: 'transparent'

  });

}

async function drawBar() {

  const res = await fetch("{ { url_for('chart_bar') } }");

  const json = await res.json();

  const data = google.visualization.arrayToDataTable(json.data);

                                                                    new

  google.visualization.ColumnChart(document.getElementById('barChart')).draw(data, {

    colors: ['#e74c3c'], legend: { position: 'none' },

    chartArea: { width: '88%', height: '72%' }, backgroundColor: 'transparent',

    vAxis: { format: '₹#,###' }

  });

}

</script>

{% endblock %}

```

Expense_form.html

```
{% extends 'base.html' %}
```

```
{% block title %}{{ action }} Expense{% endblock %}
```

```
{% block content %}
```

```
<div class="row justify-content-center">
```

```
<div class="col-md-6">
```

```
<div class="card p-4">
```

```
<h5 class="fw-bold mb-4"><i class="fas fa-plus-circle me-2 text-  
danger"></i>{{ action }} Expense</h5>
```

```
<form method="POST">
```

```
{{ form.hidden_tag() }}
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.amount.label.text }}</label>
```

```
{{ form.amount(class="form-control", placeholder="0.00") }}
```

```
{% for e in form.amount.errors %}<div class="text-danger small">{{ e  
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.category_id.label.text
}}</label>
```

```
{{ form.category_id(class="form-control") }}
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.date.label.text }}</label>
```

```
{{ form.date(class="form-control") }}
```

```
{% for e in form.date.errors %}<div class="text-danger small">{{ e
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.description.label.text
}}</label>
```

```
{{ form.description(class="form-control", rows=2) }}
```

```
</div>
```

```
<div class="d-flex gap-2 mt-3">
```

```
{{ form.submit(class="btn btn-danger flex-grow-1") }}
```

```
<a href="{{ url_for('expenses') }}" class="btn btn-outline-
secondary">Cancel</a>
```

</div>

</form>

</div>

</div>

</div>

{% endblock %}

Expenses.html

{% extends 'base.html' %}

{% block title %}Expenses{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

<h4 class="fw-bold mb-0"><i class="fas fa-money-bill-wave me-2 text-danger"></i>Expenses</h4>

<i class="fas fa-plus me-1"></i>Add Expense

</div>

<!-- Filters -->

<div class="card p-3 mb-4">

```

<form method="GET" class="row g-2 align-items-end">

<div class="col-md-4">

<label class="form-label small">Category</label>

<select name="category" class="form-control form-control-sm">

<option value="">All Categories</option>

{% for c in categories %}

    <option value="{{ c.id }}" {% if request.args.get('category') == c.id|string
%}selected{% endif %}>{{ c.name }}</option>

{% endfor %}

</select>

</div>

<div class="col-md-3">

<label class="form-label small">From</label>

<input type="date" name="date_from" class="form-control form-control-sm"
value="{{ request.args.get('date_from', '') }}">

</div>

<div class="col-md-3">

<label class="form-label small">To</label>

```

```
        <input type="date" name="date_to" class="form-control form-control-sm"
value="{{ request.args.get('date_to', '') }}">
```

```
    </div>
```

```
    <div class="col-md-2">
```

```
        <button type="submit" class="btn btn-sm btn-outline-danger w-
100">Filter</button>
```

```
    </div>
```

```
</form>
```

```
</div>
```

```
<div class="card">
```

```
    <div class="card-body p-0">
```

```
        <table class="table table-hover mb-0">
```

```
            <thead class="table-dark">
```

```
                <tr><th>Date</th><th>Category</th><th>Amount</th><th>Description</th>
><th>Actions</th></tr>
```

```
            </thead>
```

```
            <tbody>
```

```
                {% for e in expenses %}
```

<tr>

<td>{{ e.date }}</td>

<td>

{% if e.category %}

<i class="fas {{ e.category.icon }} me-1"></i>{{ e.category.name }}

{% else %}—{% endif %}

</td>

<td class="fw-bold text-danger">-₹{{ e.amount }}</td>

<td class="text-muted small">{{ (e.description or '—')[0:40] }}</td>

<td>

<i class="fas fa-edit"></i>

<i class="fas fa-trash"></i>

</td>

</tr>

{% else %}


```

        <tr><td colspan="5" class="text-center text-muted py-4">No expenses found.
    <a href="{{ url_for('add_expense') }}">Add one!</a></td></tr>

    {% endfor %}

</tbody>

</table>

</div>

</div>

{% endblock %}

```

Login.html

```

{% extends 'base.html' %}

{% block title %}Login – Expense Tracker{% endblock %}

{% block content %}

<div class="min-vh-100 d-flex align-items-center justify-content-center"

    style="background:linear-gradient(135deg,#2c3e50,#e74c3c); margin:-24px;
padding:24px;">

    <div class="card p-4" style="width:100%;max-width:400px;">

        <div class="text-center mb-4">

            <i class="fas fa-receipt fa-3x" style="color:#e74c3c;"></i>

            <h4 class="mt-2 fw-bold">Expense Tracker</h4>

```

```
<p class="text-muted">Sign in to track your expenses</p>
```

```
</div>
```

```
<form method="POST">
```

```
{{ form.hidden_tag() }}
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.username.label.text }}</label>
```

```
{{ form.username(class="form-control", placeholder="Enter username") }}
```

```
{% for e in form.username.errors %}<div class="text-danger small">{{ e
}}</div>{% endfor %}
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label fw-semibold">{{ form.password.label.text }}</label>
```

```
{{ form.password(class="form-control", placeholder="Enter password") }}
```

```
{% for e in form.password.errors %}<div class="text-danger small">{{ e
}}</div>{% endfor %}
```

```
</div>
```

```
{{ form.submit(class="btn btn-danger w-100 py-2 mt-2") }}
```

```
</form>
```

```
<p class="text-center mt-3 text-muted">No account? <a href="{ {
url_for('register') }}">Register here</a></p>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

Register.html

```
{% extends 'base.html' %}
```

```
{% block title %}Register – Expense Tracker{% endblock %}
```

```
{% block content %}
```

```
<div class="min-vh-100 d-flex align-items-center justify-content-center"
```

```
style="background:linear-gradient(135deg,#2c3e50,#e67e22); margin:-24px;
padding:24px;">
```

```
<div class="card p-4" style="width:100%;max-width:440px;">
```

```
<div class="text-center mb-4">
```

```
<i class="fas fa-user-plus fa-3x" style="color:#e67e22;"></i>
```

```
<h4 class="mt-2 fw-bold">Create Account</h4>
```

```
</div>
```

```
<form method="POST">
```

```
{ { form.hidden_tag() } }
```

```

{% for field in form %}

    {% if field.name != 'csrf_token' and field.name != 'submit' %}

        <div class="mb-3">

            <label class="form-label fw-semibold">{{ field.label.text }}</label>

            {{ field(class="form-control") }}

            {% for e in field.errors %}<div class="text-danger small">{{ e }}</div>{%
endfor %}

        </div>

    {% endif %}

{% endfor %}

{{ form.submit(class="btn btn-danger w-100 py-2 mt-2") }}

</form>

<p class="text-center mt-3 text-muted">Have an account? <a href="{{
url_for('login') }}">Login</a></p>

</div>

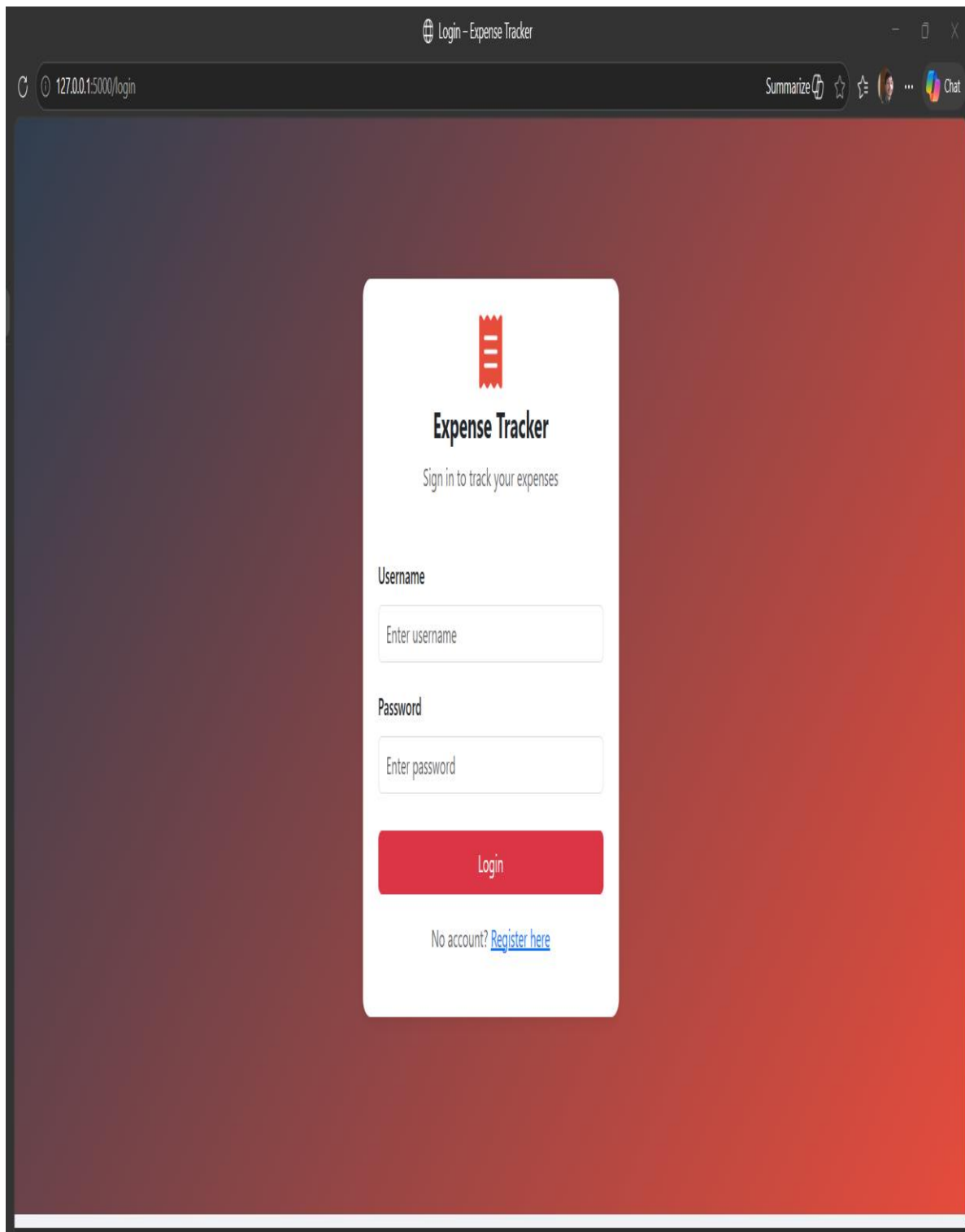
</div>

{% endblock %}

```

Screenshots

Login Page



The image shows a web browser window displaying a login page for an application titled "Expense Tracker". The browser's address bar shows the URL "127.0.0.1:5000/login". The page has a dark red gradient background. In the center, there is a white login card. At the top of the card is a red icon of a receipt. Below the icon, the text "Expense Tracker" is displayed in a bold, black font, followed by the subtitle "Sign in to track your expenses" in a smaller, gray font. The login form consists of two input fields: "Username" and "Password", each with a placeholder text "Enter username" and "Enter password" respectively. Below these fields is a red "Login" button. At the bottom of the card, there is a link that says "No account? Register here".

Login - Expense Tracker

127.0.0.1:5000/login

Summarize

Chat



Expense Tracker

Sign in to track your expenses

Username

Password

Login

No account? [Register here](#)

Register page

Register - Expense Tracker

127.0.0.1:5000/register

Summarize ☆ ⋮ Chat



Create Account

Username

Email

Preferred Currency

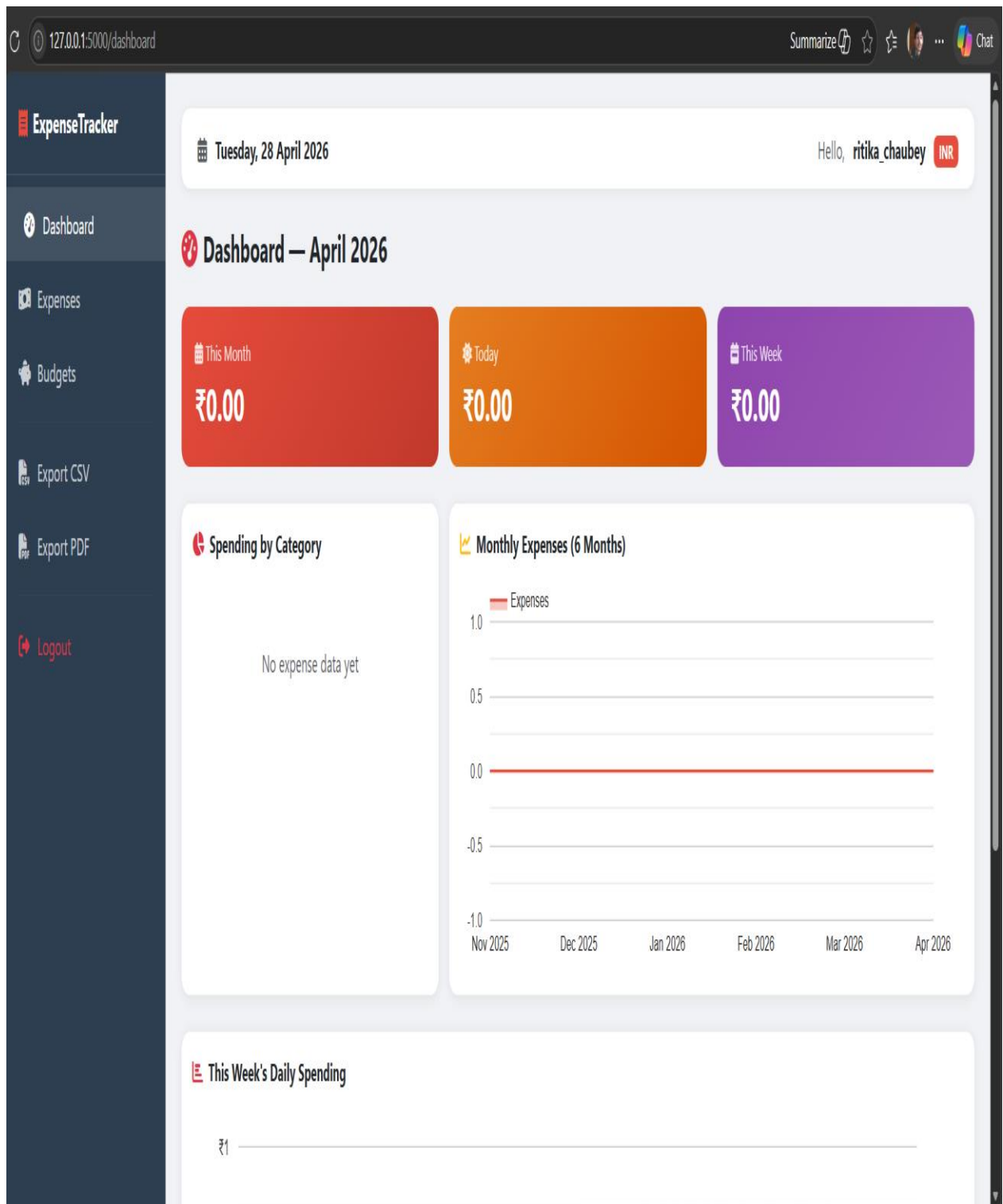
Password

Confirm Password

Create Account

Have an account? [Login](#)

Home page



Expenses page

127.0.0.1:5000/expenses

Chat

ExpenseTracker

Dashboard

Expenses

Budgets

Export CSV

Export PDF

Logout

Tuesday, 28 April 2026

Hello, ritika_chaubey

INR

Expenses

+ Add Expense

Category

From

To

All Categories

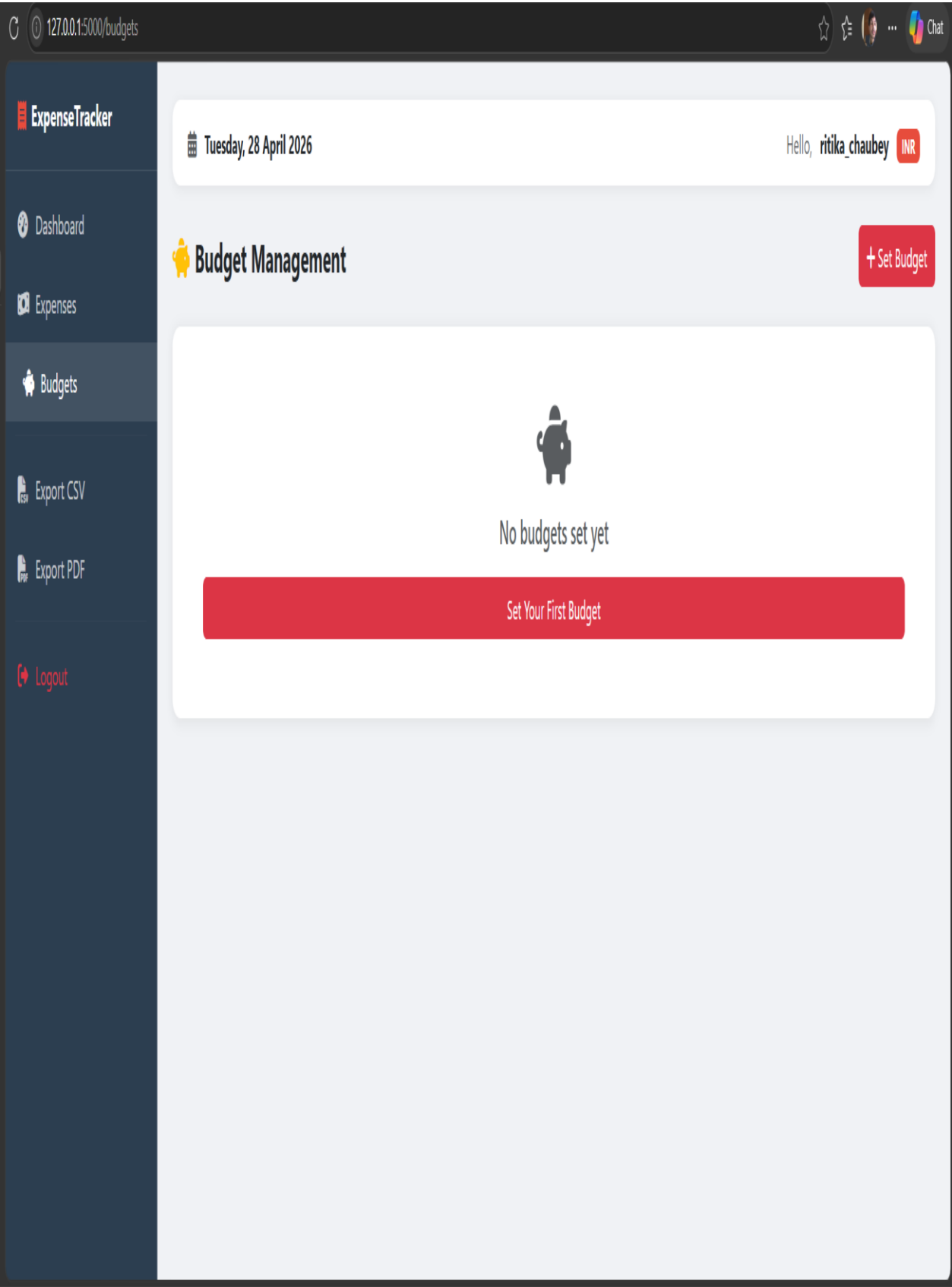
mm/dd/yyyy

mm/dd/yyyy

Filter

Date	Category	Amount	Description	Actions
No expenses found. Add one!				

Budget Page



Bibliography

Bibliography

1. Python Programming Language Official Documentation. Available at: <https://www.python.org/doc/>
(Used for understanding Python basics, functions, and libraries.)
2. Flask Documentation. Available at: <https://flask.palletsprojects.com/>
(Used for building the backend and handling web requests.)
3. SQLite Documentation. Available at: <https://www.sqlite.org/docs.html>
(Used for database design and data storage.)
4. Bootstrap Documentation. Available at: <https://getbootstrap.com/docs/>
(Used for designing responsive user interface.)
5. Google Charts Documentation. Available at: <https://developers.google.com/chart>
(Used for creating charts and graphs in dashboard.)
6. SQL Alchemy Documentation. Available at: <https://docs.sqlalchemy.org/>
(Used for database interaction in the project.)
7. WT Forms Documentation. Available at: <https://wtforms.readthedocs.io/>
(Used for form validation and user input handling.)
8. Jinja Documentation. Available at: <https://jinja.palletsprojects.com/>
(Used for rendering dynamic HTML pages.)
9. Visual Studio Code Documentation. Available at: <https://code.visualstudio.com/docs>
(Used as the development environment for coding.)
10. Tutorials and Learning Resources:
 - W3Schools. Available at: <https://www.w3schools.com/>
 - Geeks for Geeks. Available at: <https://www.geeksforgeeks.org/>
(Used for basic understanding and examples during development.)